CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In a large and dynamic educational institution like Kongu Engineering College (KEC) managing the daily volume of student inquiries regarding examinations, fee payments and administrative requests is a highly complex operational challenge Traditional support methods which rely on manual email sorting or physical office visits are time-consuming and often result in delayed resolutions and disjointed student data. The Student Help Desk System is a digital transformation project built on the industry leading Salesforce Service Cloud platform Its primary objective is to replace these inefficient manual workflows with a centralized 24/7 automated Customer Relationship Management (CRM) solution that seamlessly bridges the communication gap between the student body and college administration.

At the core of this project is an intelligent zero-touch ticketing mechanism powered by Salesforce automation Students interact with a custom-designed Web-to-Case HTML portal where they can easily submit their queries select their course (e.g., MCA, BE-CSE) and indicate the urgency of their issue. Once a ticket is submitted, the system utilizes a Master Before-Save Flow that acts as an AI-style routing engine. Instead of a staff member reading and forwarding the request this engine instantly scans the student's subject and description for specific keywords (such as "scholarship","revaluation","hall ticket") Based on these keywords the system autonomously categorizes the ticket and assigns it directly to the exact department queue such as the Controller of Examinations (COE) or the Finance Department in milliseconds.

Beyond ticket routing, the system acts as a smart self-maintaining database administrator. Using an After Save Flow the system automatically scans the incoming tickets Roll Number against the existing college database If the student is raising an issue for the first time the system dynamically generates a brand-new permanent profile for them in the custom Student object (Student__c) automatically mapping their degree course and contact details without any manual

data entry from the college staff Finally Salesforce Auto Response Rules are triggered to instantly send a branded email receipt back to the student providing them with a tracking number and ensuring a transparent, professional and highly efficient support experience.

1.2 EXISTING SYSTEM

The current student support framework at the institution is highly decentralized and relies heavily on manual processes When students face issues related to examinations, fee payments or general administration they are typically required to physically visit specific departments (such as the COE or Finance office) during restricted working hours This often involves waiting in long queues to submit paper based forms which creates significant bottlenecks during peak periods and consumes valuable time for both the student body and the administrative staff.

When digital communication is utilized it is largely unstructured and inefficient Students submit inquiries via generic emails that lack standardized formats or required details like Roll Numbers Consequently staff members are burdened with manually reading interpreting and forwarding every single message to the correct department This manual "triage" process is slow and highly prone to human error such as misrouting a fee query to the academic office Further more student records are stored in fragmented data silos like isolated Excel spreadsheets or physical ledgers meaning staff do not have immediate centralized access to a students history when trying to resolve an issue.

Crucially the existing system lacks any automated tracking or intelligent prioritization Once a student submits a request they do not receive an acknowledgment receipt or a ticket reference number leaving them unable to track the status of their grievance. Additionally because all requests pile up in a flat list urgent issues such as a failing exam portal or a time sensitive scholarship deadline are easily buried under minor general inquiries This results in a reactive support culture rather than a proactive service oriented environment.

1.3 DRAWBACKS OF EXISTING SYSTEM

Prolonged Resolution Times (High TAT) Because there is no automated routing mechanism every student query must be manually read interpreted and forwarded by administrative staff. This manual "triage" process causes severe bottlenecks often extending the Turnaround Time (TAT) for a simple issue from a few minutes to several days especially during peak academic seasons.

High Susceptibility to Human Error Depending on human operators to sort and dispatch hundreds of unstructured emails naturally leads to mistakes A critical request (such as a fee payment failure) can easily be misrouted to the wrong department accidentally deleted or buried at the bottom of a generic college inbox resulting in lost grievances.

Zero Visibility and Tracking for Students The existing framework completely lacks an automated ticketing system When students submit a physical form or send an email they do not receive an acknowledgment receipt or a unique reference number occurred.

Fragmented Data and No Single Source of Truth Student profiles, academic details, and histories of past complaints are scattered across isolated data silos such as offline Excel spreadsheets and physical paper ledgers Consequently staff members do not have immediate centralized access to a students background forcing them to manually cross reference data across multiple disconnected systems before they can even begin to solve the problem.

Inefficient Resource AllocationWithout a digital system to filter and categorize incoming requests college staff are forced to spend a disproportionate amount of their working hours performing basic clerical tasks such as sorting, reading and forwarding emails rather than actively focusing on resolving the students actual academic or financial problems.Lack of Prioritization and Escalation In a manual system, all requests pile up in a flat list.

There is no automated way to flag "High Priority" issues or alert senior management if a department is ignoring a ticket for too long. This creates a reactive environment where urgent issues are often delayed.

1.4 PROPOSED SYSTEM

To overcome the limitations of manual workflows this project proposes the Student Help Desk System a centralized CRM built on Salesforce Service Cloud Utilizing a highly efficient "Catch Route Create" methodology it replaces physical paperwork and unstructured emails with a zero touch digital workflow the core components include:

Smart Web-to-Case Portal A custom responsive HTML5/CSS3 web interface that captures structured mandatory student data (Roll Number, Course, Priority Level) to ensure no request enters the system with missing information.

Intelligent Keyword Routing (Before-Save Flow) An automated engine that scans the ticket's "Subject" and "Description" for specific keywords (e.g, "exam," "scholarship") It instantly routes the ticket to the correct department queue (COE or Fees) in milliseconds eliminating manual sorting.

Dynamic Database Management (After-Save Flow) The system cross references incoming Roll Numbers If a student does not exist in the custom database (Student__c), the system automatically creates a permanent profile for them using the web form data removing the need for manual data entry.

Automated Communication Salesforce Auto Response Rules instantly send a branded "KONGU STUDENT TICKET" email receipt. This provides students with a unique tracking number and immediate confirmation that their issue is logged.

Centralized Admin Dashboards Provides college staff with a unified view of all interactions High-priority tickets are visually flagged and real-time reports help administrators track unresolved cases and departmental performance.

Automated SLA and Escalation Tracking: Implements time-based triggers to monitor ticket resolution times Role-Based Security and Data Privacy: Leverages Salesforces robust sharing architecture to restrict data access The system ensures that sensitive information is compartmentalized allowing the Finance department to view only fee related issues and the COE to view only academic queries.

1.5 ADVANTAGES OF PROPOSED SYSTEM

Implementing the Salesforce-based Student Help Desk System offers transformative benefits for KEC Drastic Reduction in Turnaround Time (TAT) AI-style keyword routing assigns tickets to the correct department (COE or Fees) in milliseconds, bypassing manual sorting.Elimination of Human Error Automated workflows guarantee 100% accuracy ensuring tickets are never misrouted or lost in a generic inbox.Enhanced Transparency Instant "KONGU STUDENT TICKET" email receipts provide tracking numbers giving students peace of mind and eliminating physical follow ups.

Zero Manual Data Entry The system automatically builds and updates the Student__c database directly from web submissions saving staff countless hours.24/7 Accessibility The Web-to-Case portal allows students to submit high priority issues anytime anywhere.Real-Time Analytics & Accountability Salesforce Dashboards allow admins to track performance while automated SLA alerts ensure no student grievance is ignored.Eco-Friendly Ecosystem: Replaces paper forms and manual ledgers with a 100% digital sustainable workflow.

SUMMARY

The Chapter 1 established the critical need for digital transformation within Kongu Engineering College’s student support infrastructure by highlighting the severe operational bottlenecks of the existing manual grievance system. Relying heavily on physical paperwork, fragmented data silos, and disjointed email chains, the legacy approach has historically caused delayed response times, frustrated students, and a lack of administrative accountability. To resolve these systemic flaws, this chapter introduced the proposed Salesforce-based Smart Student Help Desk System as a robust, centralized CRM solution. By transitioning to this unified, cloud-native architecture, the college can successfully consolidate all departmental support operations including the Controller of Examinations (COE), Finance and IT Helpdesk under a single, highly accessible digital platform that completely replaces outdated administrative processes.

CHAPTER 2

SYSTEM ANALYSIS

2.1 IDENTIFICATION OF NEED

The initiation of the Student Help Desk System stems from the critical need to modernize student college interactions at Kongu Engineering College (KEC). As the student population continues to grow the daily volume of academic, financial and administrative inquiries has completely exceeded the processing capacity of traditional manual workflows The institution urgently needs to transition from decentralized paper based processes and unstructured emails to a unified digital platform This modernization is necessary to eliminate fragmented data silos prevent the misrouting or loss of student grievances and drastically reduce the administrative burden on college staff who currently waste hours manually sorting requests.

Technically there is an absolute necessity for an intelligent 24/7 automated Customer Relationship Management (CRM) solution The college requires a secure responsive Web-to-Case portal that allows students to log high-priority issues at any time from any device Coupled with this frontend the system needs a smart backend routing engine capable of scanning student descriptions and autonomously assigning tickets to the correct department (such as the COE or Fees queue) in milliseconds.

2.2 FEASIBILITY STUDY

A feasibility study was conducted to assess the practicality and viability of implementing the Student Help Desk System at Kongu Engineering College (KEC). The proposed system was evaluated across three primary dimensions

- Technical feasibility
- Operational feasibility
- Economic feasibility

2.2.1 TECHNICAL FEASIBILITY

The project is highly technically feasible as it is built entirely on the Salesforce Service Cloud platform a globally recognized enterprise-grade cloud architecture Operating as a Software-as-a-Service (SaaS) platform Salesforce completely eliminates infrastructure overhead meaning the college does not need to purchase, install or maintain expensive on premise servers Furthermore the system leverages low code automation utilizing Salesforces declarative Record Triggered Flows to build complex core logic like the Keyword Routing Engine and Dynamic Student Profiling without the need for extensive hard coding making the system exceptionally easy to build, test and maintain Finally the system ensures seamless integration by connecting standard frontend web technologies to the robust Salesforce backend via API guaranteeing high compatibility and a smooth user experience across all modern web browsers and mobile devices.

2.2.2 OPERATIONAL FEASIBILITY

The proposed CRM is operationally highly feasible and will drastically improve the day to day administrative workflow of the institution Because Kongu Engineering College (KEC) is an engineering institution with a highly tech savvy student body transitioning from physical paper forms to a simple responsive web portal will be a natural and frictionless process ensuring high user adoption.

Furthermore the system provides significant administrative relief for college staff across departments like the COE, Finance and General Administration by eliminating the tedious task of manually reading and sorting generic emails By automating the routing process staff can immediately focus on resolving actual grievances.

There by significantly increasing overall operational efficiency and employee satisfaction Finally the system is highly scalable as the college expands or adds new administrative branches such as Hostel or Transport departments the system can be effortlessly updated by simply adding new keywords to the existing Routing Flow without disrupting current operations.

2.2.3 ECONOMIC FEASIBILITY

The implementation of this project is economically viable and highly cost-effective for the institution By moving to a fully digital cloud based platform the college instantly reduces resource costs by eliminating the recurring expenses associated with printing physical grievance forms maintaining paper ledgers and managing physical file storage. Furthermore the system promises a high return on investment (ROI) Although there is an initial setup phase the long term financial benefits are massive the automation of ticket sorting and database creation saves thousands of administrative man hours per academic year allowing the college to optimize its workforce efficiently Finally because Salesforce offers specialized pricing and developer environments for educational implementations the licensing model remains highly affordable compared to the exorbitant costs of building a custom CRM from scratch.

2.3 SOFTWARE REQUIREMENT SPECIFICATIONS

The Automated Smart Student Help Desk System is designed as a cloud-hosted, web-based Customer Relationship Management (CRM) application. Because the heavy processing and database management are handled entirely by the Salesforce cloud infrastructure (Server-Side) the requirements for the end users (students and college staff) are minimal and highly accessible.

2.3.1 HARDWARE REQUIREMENTS

Since the application runs entirely within a web browser it does not require high end local computing power The following are the minimum hardware specifications required for students to submit tickets and for administrators to manage the Salesforce backend:

- **Processor:** Intel Core i3 / AMD Ryzen 3 (or equivalent mobile processor) and above.
- **Random Access Memory (RAM):** 4 GB minimum (8 GB recommended for seamless multitasking on the Salesforce Lightning Console).
- **Storage (Hard Drive/SSD):** Minimal local storage required (approx. 100 MB for browser caching) all system data is stored securely on the cloud.

- **Display Resolution:** 1024 x 768 pixels or higher (Responsive design adapts to mobile screens).
- **Network Connectivity:** A stable, active internet connection (Broadband, Wi-Fi, or 4G/5G mobile network) is strictly required to access the cloud CRM and submit the Web-to-Case form.

2.3.2 SOFTWARE REQUIREMENTS

The system relies on modern web technologies and the Salesforce Lightning Platform to deliver its automated workflows and user interface.

- **Operating System (Platform Independent):** Windows 10 / 11, macOS, Linux distributions, Android or iOS.
- **Supported Web Browsers:** Google Chrome (Highly Recommended) Mozilla Firefox, Apple Safari or Microsoft Edge.
- **Frontend Technologies (User Interface):** * HTML5: Used to structure the custom KEC Web-to-Case portal.
- **CSS3:** Used to style the web form provide responsive mobile scaling, and create interactive UI elements (like success pop-ups).
- **Backend Platform & Application Logic:** * Platform: Salesforce Service Cloud (Lightning Experience).
- **Automation Engine:** Salesforce Flow Builder (specifically Master Before-Save and After-Save Record-Triggered Flows).
- **Communication Rules:** Salesforce Auto-Response Rules and Email Templates.
- **Database & Data Management:**
- **Database Technology:** Salesforce Proprietary Multi-Tenant Cloud Database.
- **Standard Objects Used:** Case (for ticketing) Group (for Queues).
- **Custom Objects Used:** Student__c (for the student directory), Routingrule__c (for the keyword engine).

2.4 SOFTWARE DESCRIPTION

2.4.1 FRONT END(Web-to-Case Portal)

The front end of the Student Help Desk System serves as the primary interactive layer between the student body and the colleges backend CRM Built utilizing modern HTML5 and CSS3 this custom Web-to-Case portal is designed to be highly intuitive visually aligned with Kongu Engineering Colleges branding and fully responsive across all mobile and desktop devices Unlike generic unstructured email platforms this dedicated web interface enforces structured data collection.

It features specifically tailored input fields such as the students Roll Number, Contact Details, Degree Course (e.g., MCA, BE-CSE via custom dropdowns) and the Priority Level of their grievance By applying strict HTML validation and required field constraints the front end ensures that no incomplete or ambiguous request is ever submitted thereby providing a seamless error free user experience for the students.

From a technical integration perspective the front end acts as a highly secure real-time data bridge directly into the Salesforce Service Cloud The HTML form utilizes the HTTP POST method to securely transmit the captured student data to the proprietary Salesforce Web-to-Case servlet Within the frontend code critical hidden elements such as the unique Salesforce Organization ID (orgid) are embedded to ensure the data is routed precisely to the college's specific cloud instance.

Furthermore custom data points like the 'Student Course' and 'Roll Number' are intricately mapped using unique Salesforce API alphanumeric identifiers (e.g, 00N...) allowing the front end to communicate seamlessly with the backend custom objects. Once the student clicks the submit button the front end instantly packages this structured information and pushes it into the cloud database immediately triggering the backend routing flows without requiring any third-party middleware.

2.4.2 BACK END(Salesforce Service Cloud & Automation)

The backend infrastructure of the Automated Smart Student Help Desk System is powered entirely by the Salesforce Service Cloud a robust multi-tenant enterprise CRM platform This cloud environment acts as the central repository and processing hub for all student grievances At its core is a highly relational database architecture utilizing both Standard Objects (such as the Case object for ticketing and tracking) and Custom Objects (such as Student__c for the student directory and Routingrule__c for keyword logic) When the Web-to-Case frontend transmits its payload the Salesforce backend securely intercepts this data and seamlessly translates it into structured records This cloud native approach ensures that Kongu Engineering College administrators have real-time, secure and 24/7 access to a centralized "single source of truth" entirely eliminating the need for local server maintenance or manual ledger updates.

Beyond simple data storage the true power of this backend lies in its intelligent, low-code automation layer, driven by Salesforce Record-Triggered Flows The system employs a sophisticated "Master Before-Save" flow that functions as an automated keyword routing engine The exact millisecond a ticket enters the database this flow scans the students subject and description for specific trigger words (e.g, "scholarship" "exam" "revaluation") and autonomously assigns

The ticket to the appropriate departmental queue (Fees or COE) without any human intervention Immediately following this an "After-Save" flow acts as a dynamic database manager; it cross-references the incoming Roll Number and automatically generates a permanent student profile if one does not already exist Together these backend automation tools ensure unprecedented speed, 100% routing accuracy and a zero-touch administrative workflow.

2.4.3 DATABASE (Salesforce Cloud Data Architecture)

The database infrastructure for the Automated Smart Student Help Desk System is built entirely on Salesforces proprietary multi-tenant cloud architecture Unlike traditional on premise Relational Database Management Systems (RDBMS) such as MySQL or Oracle the Salesforce database operates as a secure highly scalable Database-as-a-Service (DBaaS) This cloud native approach completely eliminates the need for the college IT department to manage server provisioning manual data backups or database software updates

The data model is intrinsically relational and highly customizable utilizing a combination of Standard Objects which come pre built within the Salesforce CRM to handle standard service operations and Custom Objects which are specifically engineered from scratch to meet the unique data tracking requirements of Kongu Engineering College.

Within this architecture the system relies on a specific data schema to seamlessly link student identities with their respective grievances The core transactional entity is the standard Case object which securely stores every incoming Web-to-Case submission including the subject grievance description, priority level and the assigned department queue. To maintain an accurate self-updating student directory a custom Student__c object was developed. This acts as the master record for each student storing unique identifiers like their Roll Number and Degree Course.

A critical feature of this database design is its dynamic relational capability when a new Case is submitted backend automation automatically establishes a relational link (Lookup) between the Case and the Student profile Furthermore a custom Routingrule__c object functions as a specialized configuration table storing the exact text keywords (e.g., "exam," "fees") and their corresponding queue assignments acting as the foundation for the AI-style routing engine.

SUMMARY

Chapter 2 presented a comprehensive system analysis for the Student Help Desk System detailing the technical foundations required to replace the existing manual workflows at Kongu Engineering College It established the critical need for a centralized 24/7 digital CRM to handle the growing volume of student grievances A detailed feasibility study confirmed that leveraging the Salesforce Service Cloud is technically robust operationally highly efficient for administrative staff and economically viable due to the complete elimination of on premise server costs and paper based processes.

Furthermore the chapter outlined the specific hardware and software requirements necessary for successful deployment defining the software architecture as a multitier cloud application consisting of a responsive HTML5 and CSS3 frontend portal (Web-to-Case) seamlessly integrated with the powerful Salesforce backend.

CHAPTER 3

SYSTEM DESIGN

3.1 MODULE DESCRIPTION

To ensure high scalability system maintainability and seamless execution the Automated Smart Student Help Desk System is divided into five distinct highly integrated functional modules Each module is engineered to handle a specific phase of the ticketing lifecycle progressing from initial data capture to final resolution and administrative reporting.

3.1.1 User Interface & Data Capture Module (Web-to-Case)

This frontend module serves as the primary touchpoint for the student body Its primary objective is to replace unstructured handwritten grievance letters with a standardized digital format.

Technical Implementation: Built using responsive HTML5 and CSS3 the web form adapts perfectly to mobile screens allowing students to submit queries on the go The form employs strict client-side validation (using HTML attributes like required and pattern matching) to ensure that crucial data such as a valid 8-digit Roll Number, Department Picklist selection (e.g, BE-CSE, B.Tech-IT) and Priority Level is captured accurately before submission.

Data Transmission: Under the hood the module uses a secure HTTP POST request targeting the Salesforce Web-to-Case API servlet It utilizes hidden input fields containing the colleges unique Salesforce Organization ID (orgid) which securely authenticates the payload and guarantees the data is routed precisely to the institutions private cloud instance without any intermediary middleware.

3.1.2 Intelligent Routing & Triage Module (Before-Save Flow)

This is the core automation engine of the system designed to completely replace manual email sorting.

It utilizes a Salesforce Master Before-Save Record-Triggered Flow The precise millisecond a ticket enters the system this module scans the textual data in the "Subject" and "Description" fields.

Based on predefined keywords stored in the Routingrule__c object (e.g, matching "exam" to the COE Queue or "scholarship" to the Finance Queue) it autonomously assigns the ticket to the correct departmental queue before the record is saved to the database.

3.1.3 Dynamic Profile Management Module (After-Save Flow)

This backend database management module acts as a self maintaining digital archivist It ensures that the colleges student directory is always up-to-date without requiring staff to manually type in student details.

Technical Implementation: Triggered immediately after the Case record is successfully saved, this module executes an After-Save Record-Triggered Flow. It performs a database query (SOQL equivalent) to cross-reference the incoming tickets Roll Number against the existing custom Student__c object records.

If the system finds a matching Roll Number it automatically establishes a relational link (Lookup Relationship) between the new Case and the existing Student profile If no match is found (indicating a new student or first-time user) the module executes a DML (Data Manipulation Language) Insert operation It dynamically creates a brand new permanent student profile using the Name, Course and Contact details captured from the web form thereby completely automating the data entry process.

3.1.4 Automated Communication Module

This module is dedicated to establishing a professional, transparent and immediate feedback loop with the student solving the "communication black hole" prevalent in manual systems.

Implementation: Driven by Salesforce Auto-Response Rules and customized HTML Email Templates.

The moment a case is logged and routed this module triggers an outgoing email back to the students provided email address. The email template uses dynamic merge fields (e.g, Dear {!Student__c.Name} your ticket {!Case.CaseNumber} has been received) to personalize the response This branded "KONGU STUDENT TICKET" receipt provides a unique tracking number assuring the student that their grievance is officially registered and actively being monitored by the respective department.

3.1.5 Administrative Dashboard & SLA Monitoring Module

This backend interface provides the management and administrative staff with the tools necessary to monitor, prioritize and resolve the routed tickets efficiently.

Technical Implementation: This module leverages the Salesforce Lightning App Builder to create role-specific workspaces It utilizes List Views, Custom Reports and dynamic Dashboards.

Operational Logic: Department heads (e.g., the Head of the COE) log into a centralized console where high-priority tickets are visually flagged in red Furthermore the module enforces operational accountability through Service Level Agreements (SLAs). Using time-dependent workflow actions the system continuously monitors the "Age" of open cases. If a department fails to resolve a critical ticket within a predefined timeframe (e.g, 48 hours) the module automatically triggers an escalation rule sending an alert to higher management (like the Principal or Admin) to prevent unresolved student issues.

3.1.6 Reports and Dashboard Module

This final module serves as the analytical powerhouse of the system, transforming raw ticket data into actionable insights for the college administration Its primary goal is to shift the institution from a reactive problem-solving approach to a proactive data-driven management style.

Technical Implementation: This module is built using the native Salesforce Report Builder and Dashboard Builder eliminating the need for third-party Business Intelligence (BI) tools It utilizes Summary and Matrix report formats to group data by specific parameters (e.g, grouping Cases by Department__c or Status) The Dashboard component then translates these reports into real-time visual metrics using dynamic UI elements like pie charts (for ticket distribution) bar graphs (for ticket volume over time) and gauge charts (to measure target resolution times) Furthermore the module enforces Role-Based Access Control (RBAC) ensuring that the Finance HOD only sees finance-related analytics while the College Principal has a macro level view of all departments.

Operational Logic: In day-to-day operations this module allows management to monitor Key Performance Indicators (KPIs) at a glance Administrators can track the total volume of incoming requests compare the workload between the COE and Fees departments and identify recurring academic issues (e.g, a sudden spike in "Hall Ticket" queries before exam week) By visualizing the Turnaround Time (TAT) and highlighting unresolved "High Priority" tickets this module empowers the administration to allocate staff resources more effectively and ensure strict compliance with institutional support standards.

3.2 DATA FLOW DIAGRAM (DFD)

A Data Flow Diagram (DFD) is a graphical representation used to map the flow of information for the Student Help Desk System It illustrates how student grievance data enters the system how it is transformed by the Salesforce automation logic where it is stored in the database and where it is eventually routed.

Standard DFD Notations Used:

- **External Entity (Square):** Represents outside sources or destinations of data (e.g., Student, Department Head).
- **Process (Circle or Rounded Rectangle):** Represents a function that transforms the data (e.g, Keyword Routing, Data Validation).
- **Data Store (Open-ended Rectangle):** Represents the database where data is held (e.g, Salesforce Case Object, Student__c Object).
- **Data Flow (Arrow):** Shows the direction of data movement between entities, processes and data stores.

3.2.1 DFD Level 0

The Level 0 Data Flow Diagram commonly referred to as the Context Diagram represents the highest level of abstraction for the Student Help Desk System. At this level the entire Salesforce based application is depicted as a single centralized process The primary objective of this diagram is to establish the overall system boundary clearly identifying the external users who interact with it and mapping the precise data streams flowing in and out of the system environment without getting bogged down by internal technical logic.

Process 0: Student Help Desk System

Description: This central node represents the core Salesforce CRM application operating as a functional "black box" It encapsulates all backend operations including the Web-to-Case data capture the Before-Save keyword routing engine and the relational database commits It serves as the intelligent automated intermediary between the student body and the college administration.

External Entity 1: Student

Role: The student is the primary initiator and the main source of incoming data for the system.

Inbound Data Flow (To System): The student interacts with the frontend web portal to submit a structured data payload This outbound flow from the student includes critical

identifiers such as their Roll Number and Course alongside the operational data like the Query Description and the perceived Priority Level (e.g., High, Medium, Low).

Outbound Data Flow (From System): To ensure complete transparency and solve the "communication black hole" of manual system the system immediately returns a data flow back to the student This consists of an Automated Email Confirmation containing a unique auto-generated Ticket Tracking Number assuring the student that their grievance is officially registered.

External Entity 2: **Department Administrator (COE / Finance)**

Role: The administrative staff members act as the resolving entities They rely on the system to filter, organize and prioritize the massive influx of student issues.

Outbound Data Flow (From System): The system pushes processed and heavily organized data to the administrators This flow includes the seamlessly Assigned Tickets routed strictly to their specific departments queue, Escalation Alerts for cases nearing their resolution deadlines and comprehensive Dashboard Analytics that provide a macro level view of departmental performance.

Inbound Data Flow (To System): As the administrator actively works on resolving a grievance they feed operational data back into the CRM This vital flow includes Case Status Updates (e.g, shifting a case from "New" to "Closed") and internal Resolution Comments which are permanently appended to the database to maintain a historical audit trail.

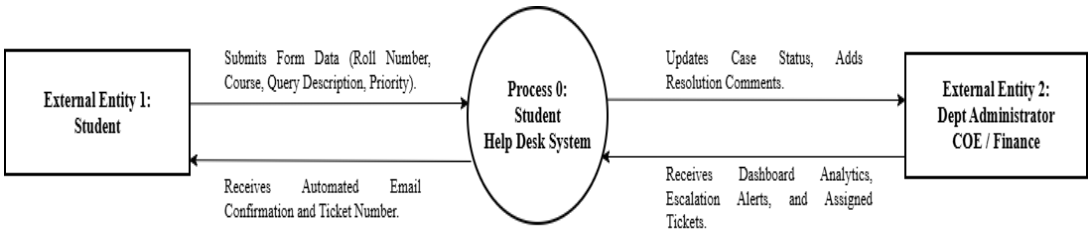

Figure 3.1 Data Flow Diagram Level 0

3.2.2 DFD Level 1

The Context Diagram illustrates the system as a single entity the Level 1 Data Flow Diagram "opens the black box" to reveal the internal architecture of the Salesforce CRM It

decomposes Process 0 into five distinct sequential sub processes illustrating exactly how data is transformed where the automation logic is applied and how information is permanently written to the internal Data Stores.

Data Stores Used in Level 1: Before data reaches the end users it interacts with three primary backend databases

D1: Case Data Store: The standard Salesforce Case object that holds all ticket details (Status, Priority, Description)

D2: Student Data Store: The custom Student__c object that acts as the master directory for student profiles.

D3: Keyword Rules Store: The custom Routingrule__c object holding the conditional logic (e.g., matching "exam" to the COE).

Process 1.0: Data Capture (Web-to-Case)

Description: This process acts as the secure gateway into the system It receives the raw unstructured inputs from the Student entity via the web portal.

Data Flow: It validates the HTML form data (ensuring mandatory fields like Roll Number and Course are filled) and transforms it into a secure HTTP POST payload This payload is then transmitted directly into the Salesforce cloud environment ready for logical processing.

Process 2.0: AI Keyword Routing (Before-Save Flow)

Description: Functioning as the intelligent "brain" of the system this process intercepts the raw text payload before it is saved to the database.

Data Flow: It pulls reference data from the D3 Keyword Rules Store By scanning the students "Subject" and "Query Description" for these specific trigger words (e.g, "fees", "hall ticket") this process dynamically assigns the correct OwnerId (departmental queue) to the data packet ensuring 100% routing accuracy without human intervention.

Process 3.0: Database Commit & Profiling (After-Save Flow)

Description: This is the core archival process where temporary data becomes permanent institutional data.

Data Flow: First the routed ticket is officially committed to the D1 Case Data Store Instantly an After Save automation flow pulls the students Roll Number from the payload and queries the D2 Student Data Store If a match is found it links the new ticket to the existing profile If no match exists this process autonomously generates a brand new student profile in D2 eliminating manual data entry for the college staff.

Process 4.0: Auto Notification Rule

Description: This process handles the outbound communication loop to maintain transparency with the student body.

Data Flow: Triggered by a successful write operation in Process 3.0 this rule pulls the newly generated Ticket Tracking Number from D1 and the corresponding students email address from D2 It combines these into a dynamic HTML email template and pushes the "Ticket Created" confirmation back to the Student entity.

The Trigger (Event Listener): The rule remains on standby until it detects a successful "write" operation from Process 3.0 (which handles the initial ticket creation and AI routing). The exact millisecond the database confirms the new ticket is saved Process 4.0 is activated.

Data Extraction (D1 & D2 Integration): Once triggered the system runs a query across two distinct data stores It queries D1 (The Ticket/Case Database) to retrieve the newly generated, unique Ticket Tracking Number and the Grievance Subject Simultaneously it queries D2 (The Student Profile Database) to extract the specific student's verified Email Address and First Name.

Dynamic Synthesis (Template Assembly): The system then feeds this extracted data into a pre-configured HTML email template. Using dynamic merge fields (e.g., {!Case.CaseNumber}) the system seamlessly embeds the student's exact tracking details into the standard boilerplate text, personalizing the message instantly.

Output Dispatch (Push to Entity): Finally the system utilizes the outbound email server to push the "Ticket Created" confirmation back to the external Student Entity.

Process 5.0: Admin Dashboard & Resolution

Description: This process provides the visual management interface utilized by the Department Administrators.

Data Flow: It continuously queries both D1 Case Data and D2 Student Data to generate real time metrics lists of assigned tickets and SLA escalation alerts pushing this visualized data to the Administrator entity Conversely when the Administrator resolves an issue they input "Case Status Updates" and "Resolution Comments" which this process securely writes back into the D1 Data Store to maintain a permanent audit trail.

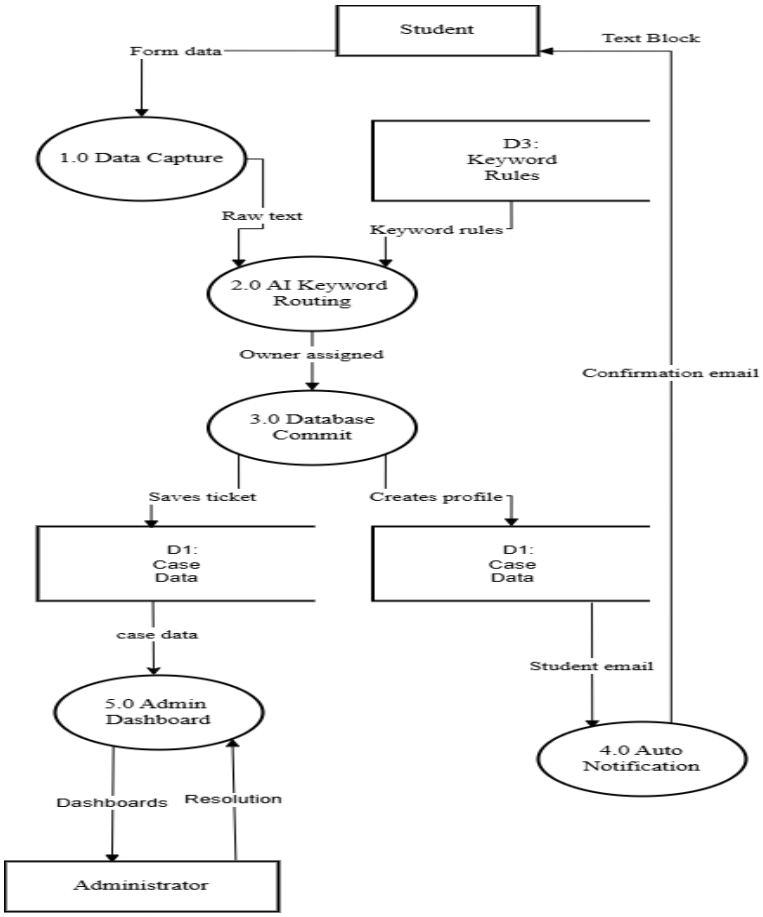

Figure 3.2 Data Flow Diagram Level 1

3.3 USE CASE DIAGRAM

A Use Case Diagram is a behavioral Unified Modeling Language (UML) diagram that provides a high level visual overview of the systems functional requirements For the Student Help Desk System this diagram maps out the interactive boundaries between the human users (Actors) and the automated capabilities of the Salesforce CRM (Use Cases) It illustrates what actions each user is permitted to perform within the system without detailing the complex underlying logical sequences.

1. The Actors (Who uses the system):

- **Student:** The primary initiator of the process They interact strictly with the external facing web portal and their personal email.
- **Department Administrator (COE / Finance):** The operational staff responsible for resolving grievances They work inside the secure Salesforce Lightning console.
- **System Admin:** The high level technical overseer who manages the platforms configuration, rules and user access.
- **Salesforce Engine (System Actor):** Represents the automated backend workflows (Flows and Triggers) that autonomously execute tasks without human intervention.

2. The Use Cases (What the system does):

The proposed system architecture is driven by well-defined user interactions, prioritizing the student as the core end-user. The primary use cases for the student entity encompass the initiation of support requests and the receipt of system-generated acknowledgments. First, students interact with the platform to Submit Grievance Forms. This is facilitated through a secure, structured frontend portal (such as the Web-to-Case interface), which replaces unstructured emails and physical paperwork with a standardized digital intake process. Second, students Receive Ticket Confirmations. The exact moment a grievance is successfully logged into the Salesforce database, the system autonomously triggers an outbound communication protocol.

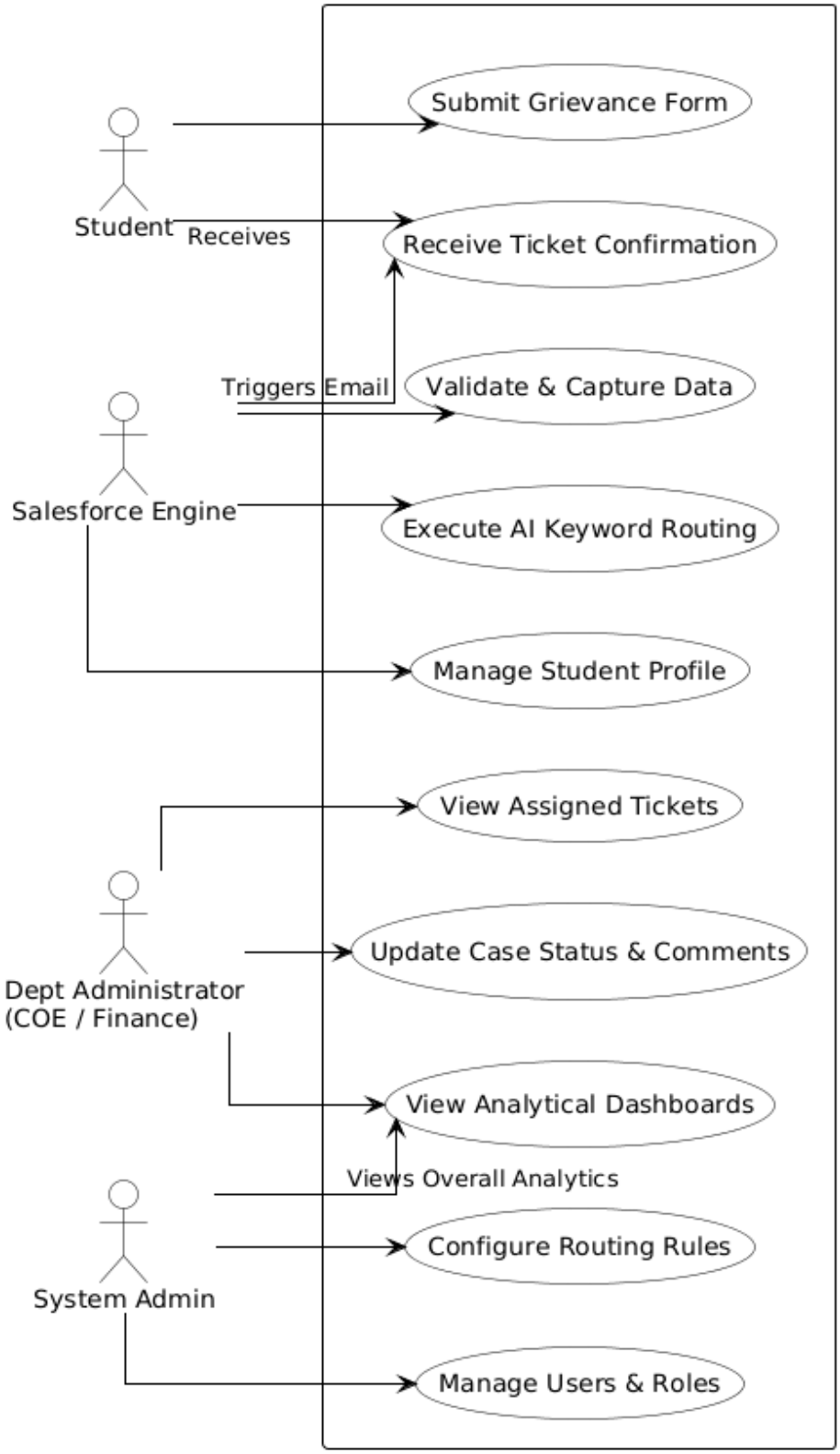

Figure 3.3 UseCase Diagram

3.4 INPUT DESIGN

Input design is the process of creating user-friendly interfaces that facilitate the accurate, secure and structured entry of data into the system. For the Automated Smart Student Help Desk System the input design focuses primarily on the student facing frontend specifically the Web-to-Case HTML portal.

The primary objective of this input design is to replace unstructured handwritten grievance letters with a standardized digital format This ensures that the Salesforce backend receives high quality pre formatted data which is absolutely necessary for the AI keyword routing engine to function correctly.

Student help desk form:

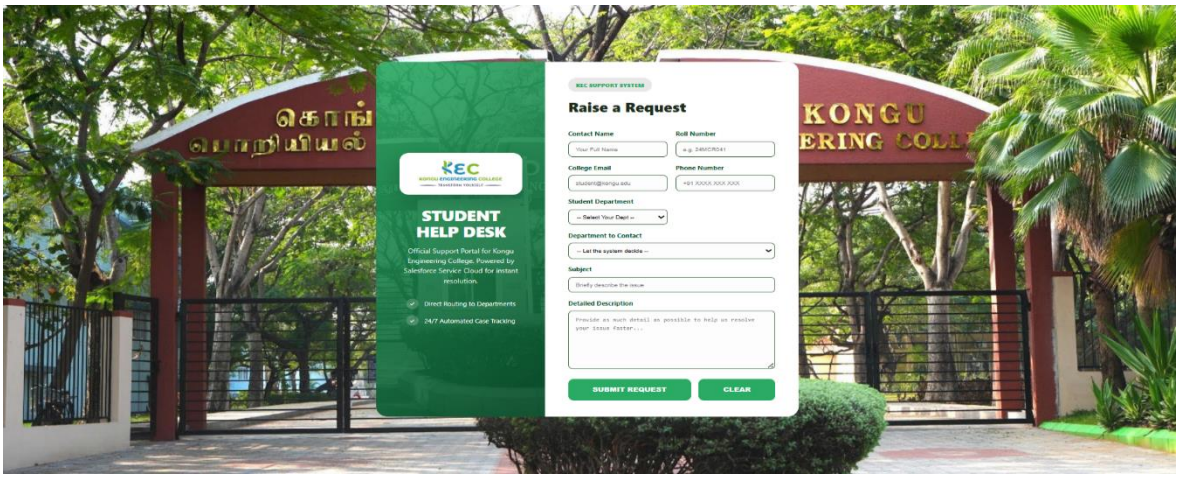

Figure 3.4 Student help desk form

The input design for the Automated Smart Student Help Desk System is realized through a custom-built, responsive Web-to-Case portal that serves as the primary gateway for logging student grievances. As illustrated in Figure 3.4.1 this institutionally branded interface is engineered to capture standardized high-quality data by requiring essential fields such as the student's Contact Name, Roll Number, College Email and Department alongside expansive text areas for the Subject and Detailed Description.

By enforcing strict data validation at the frontend the system completely eliminates unstructured, handwritten requests and ensures that the Salesforce backend receives the exact parameters and text keywords necessary to autonomously create student profiles and trigger the AI-driven departmental routing rules without any manual data entry.

Ticket form submitted:

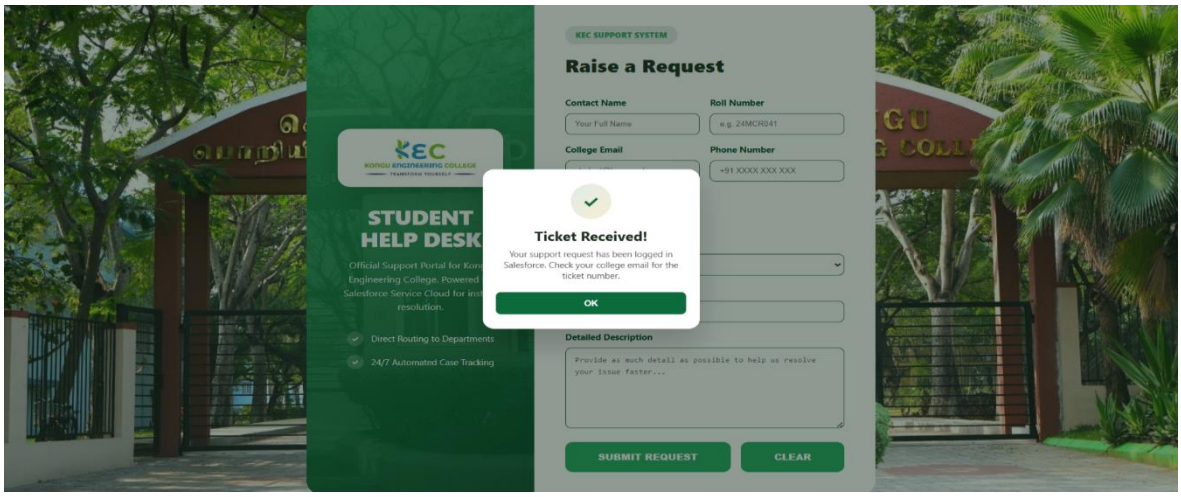

Figure 3.5 Ticket form submitted

As illustrated in Figure 3.4.2 upon the successful submission of the grievance form, the frontend interface immediately presents a real time "Ticket Received" confirmation modal to the user This visual feedback mechanism is triggered the exact moment the data payload is securely transmitted and logged into the Salesforce backend It serves a crucial user experience (UX) function by instantly assuring the student that their issue has been successfully registered without any transmission errors Furthermore the notification proactively instructs the student to check their provided college email address creating a seamless transition between the frontend input phase and the systems automated backend communication module where Salesforce Auto Response Rules will deliver their official unique ticket tracking number.

Student Automated Ticket Confirmation mail:

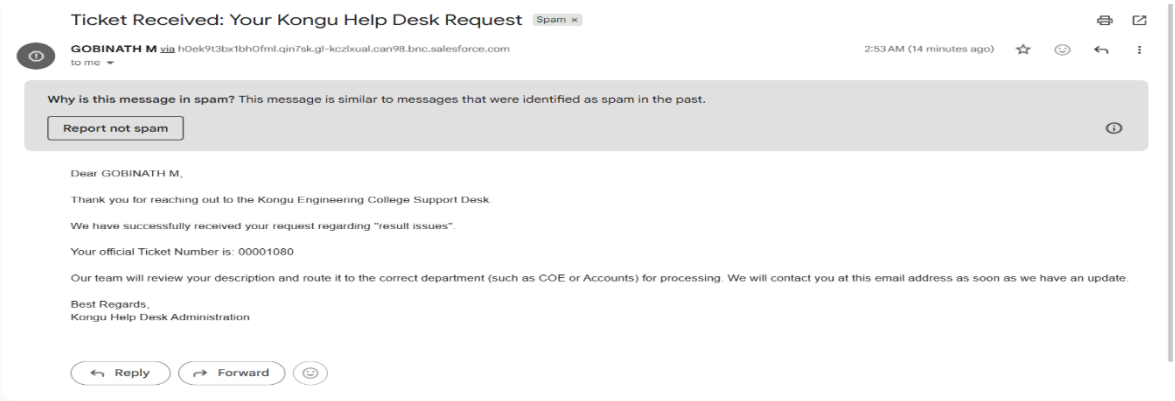

Figure 3.6 Student Automated Ticket Confirmation mail

As depicted in Figure 3.6 the systems outbound communication module utilizes Salesforce Auto Response Rules to instantly generate and dispatch a customized confirmation email to the student upon successful case creation This automated output serves as a formal digital receipt dynamically embedding specific submission details such as the students name the stated grievance subject, and most importantly a uniquely generated Ticket Tracking Number (e.g., 00001080) using backend system merge fields.

By providing immediate acknowledgment and clearly outlining the subsequent departmental routing process this feature effectively resolves the communication gap prevalent in legacy manual systems ensuring complete transparency and assuring the student that their request is officially registered and actively being processed by the administration.

COE Automated Ticket Confirmation mail:

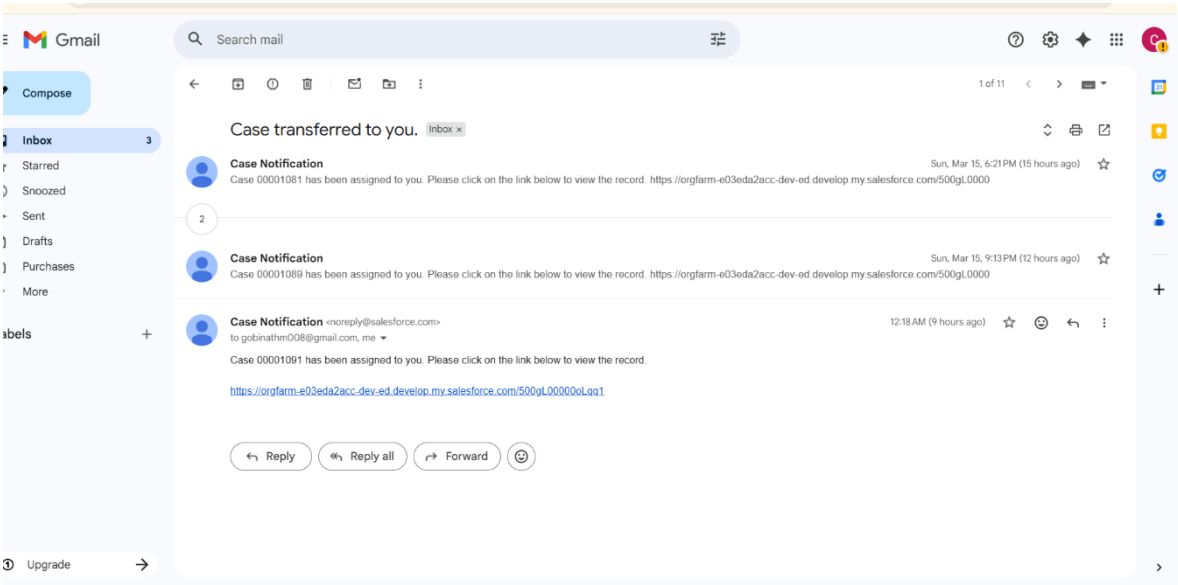

Figure 3.7 Coe Automated Ticket Confirmation mail

Fees Automated Ticket Confiramation mail:

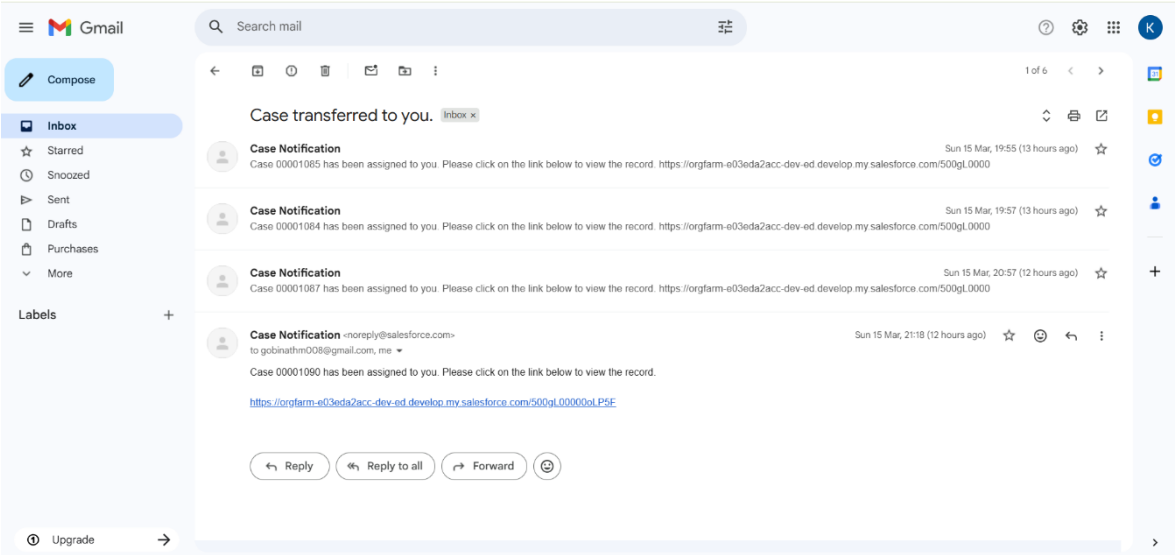

Figure 3.8 Fees Automated Ticket Confiramation mail

Case created stored in salesforce:

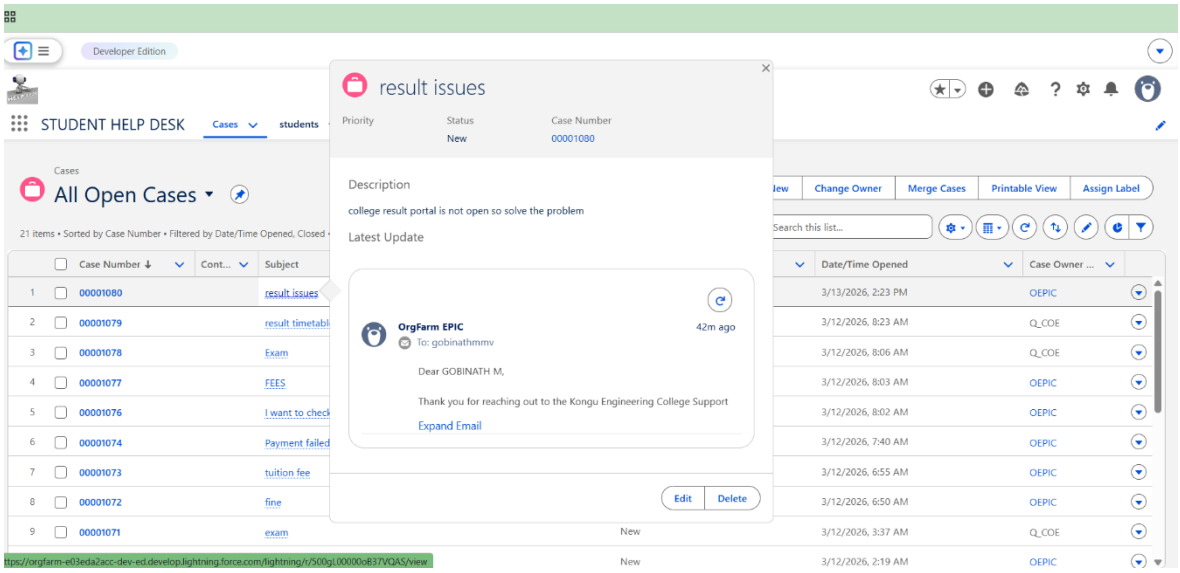

Figure 3.9 case created in stored in salesforce

Figure 3.9 illustrates the system provides a robust backend administrative dashboard built on the Salesforce Lightning Service Console, designed specifically for department staff to manage and resolve student grievances The background of the interface features a centralized list view of "All Open Cases" which automatically organizes incoming tickets by their unique Case Number, Subject and dynamically assigned Case Owner (e.g., the COE Queue) The

foreground highlights a powerful quick view modal for a specific ticket (Case 00001080) which displays the students exact issue description and provides a chronological audit trail of all interactions.

Notably this view includes the systems automated outgoing email granting administrators full visibility into the communication timeline This centralized single screen workspace eliminates data silos allowing college staff to quickly assess, claim and resolve student issues with maximum efficiency.

Student details in salesforce:

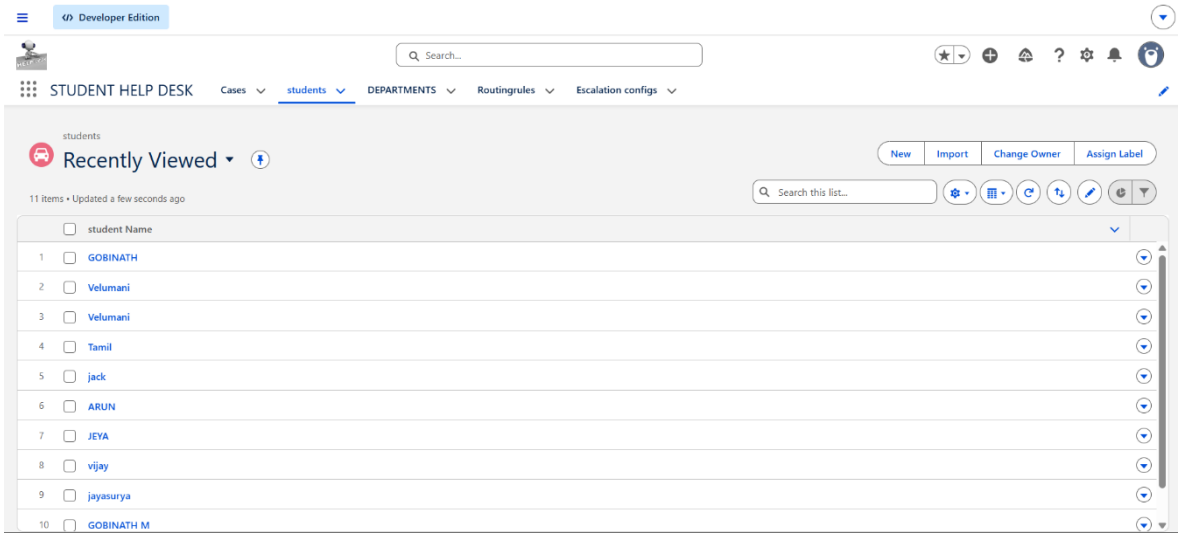

Figure 3.10 Student details in salesforce

3.5 ER DIAGRAM

Entity-Relationship (ER) Diagram The Entity-Relationship (ER) diagram models the logical database structure of the Automated Smart Student Help Desk System within the Salesforce Service Cloud environment It illustrates the primary data entities (Salesforce Objects) their attributes (Fields) and the cardinality of the relationships connecting them.

STUDENT (Student__c) This custom master entity stores demographic data. It shares a One-to-Many (1:N) relationship with the CASE entity, meaning a single student profile can be linked to multiple grievance tickets over their academic tenure, but a specific ticket belongs to only one student. This relationship is physically established via a Lookup field in Salesforce.

CASE (Standard Object): This is the central transactional entity representing the grievance ticket (e.g., Ticket 00001080). It inherits foreign keys (FK) from both the STUDENT (who raised it) and the DEPARTMENT_QUEUE (who owns it).

DEPARTMENT_QUEUE (OwnerId): Represents the administrative division (e.g., COE or Finance) responsible for resolving the issue. It shares a One-to-Many (1:N) relationship with the CASE entity, as one department manages a massive queue of many tickets.

ROUTING_RULE (Routingrule__c): This custom entity stores the logic framework (Keywords) used by the system. It relates to the queues, dictating which department receives the ticket if a specific keyword is matched during the before-save flow.

The primary use of an Entity-Relationship (ER) diagram in the system design phase is to serve as a foundational structural blueprint for the underlying database architecture For the Automated Smart Student Help Desk System, it visually maps out the core data entities such as Students, Cases (Tickets) and Administrators and explicitly defines the logical relationships, foreign keys and cardinality connecting them. In addition to mapping entities, it meticulously details the specific attributes required for each record, such as the student's roll number or the ticket's priority level.

This abstract modeling is crucial because it allows developers to identify exact data requirements prevent database redundancy and ensure referential integrity long before any physical tables are configured in the Salesforce environment Furthermore the ER diagram acts as a universal communication tool enabling both technical system architects and non-technical college stakeholders to clearly understand exactly how student grievance data is captured, stored, and related within the system. By finalizing these data connections early, the development team can optimize complex database queries, which is essential for powering the real-time reporting dashboards used by college management. Additionally, this blueprint directly guides the configuration of Salesforce-specific architectures, such as choosing between Lookup and Master-Detail relationships to enforce strict data security.

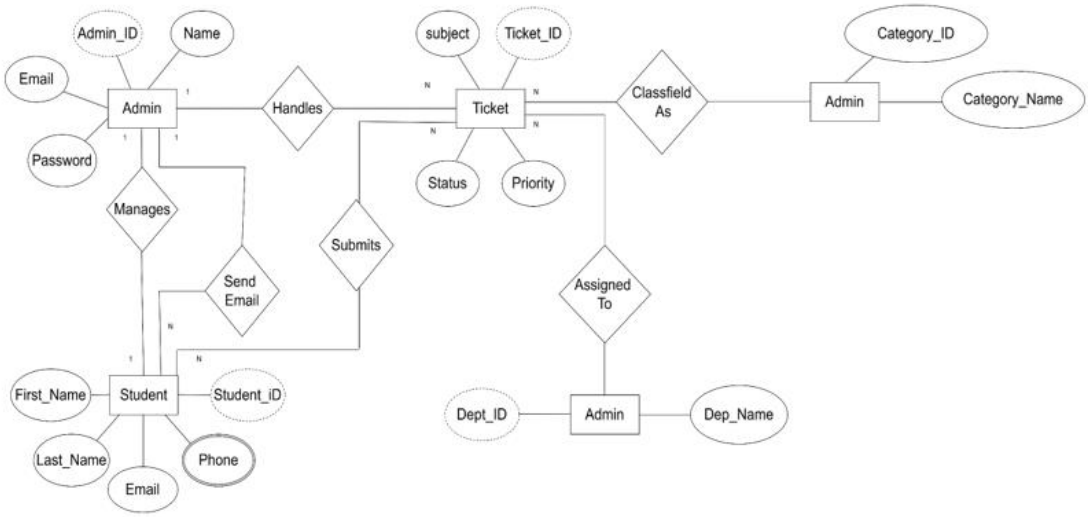

Figure 3.11 ER diagram

3.6 DATABASE DESIGN

The database architecture for the Automated Smart Student Help Desk System is built upon Salesforces robust multi-tenant cloud infrastructure utilizing an Object-Relational Database Management System (ORDBMS) model. In this environment, traditional database tables are represented as "Objects" columns as "Fields" and rows as "Records" This cloud native design ensures that Kongu Engineering College’s grievance data is stored securely with high availability eliminating the need for on premise server maintenance while providing a highly scalable foundation for managing thousands of student interactions seamlessly.

At the transactional core of this database design is the standard Salesforce Case object which acts as the primary repository for all incoming student grievances When a student submits an issue via the Web-to-Case frontend porta a new Case record is instantly instantiated in the backend capturing critical operational data such as the issues Subject Detailed Description Priority level and current Status. This object functions as the central anchor for the entire resolution lifecycle,allowing administrative queues to track, update and eventually close the ticket while maintaining a permanent time stamped audit trail of all actions and communications.

To specifically adapt the Salesforce CRM to the institutions academic environmen the database schema incorporates highly tailored custom objects most notably Student__c and Routingrule__c The Student__c object serves as the master directory for the student body storing unique identifiers like the students Roll Number, College Email and Department to ensure accurate profile management and eliminate redundant data entry for returning users Simultaneously the Routingrule__c object acts as the configuration hub for the systems automation, storing the specific keyword triggers and target queues required by the backend logic to autonomously route tickets to the correct administrative departments (such as the COE or Accounts) without any manual triage.

The structural integrity of this database is maintained through precisely defined relational mechanisms primarily utilizing Lookup relationships to seamlessly connect these distinct objects For instance a one-to-many (1:N) relationship links a single Student__c profile to multiple Case records ensuring that administrators can instantly view a students entire historical grievance timeline from a single unified interface By enforcing strict referential integrity and leveraging Salesforces relational schema the database effectively eliminates data duplication prevents orphan records and guarantees that the systems real time analytical dashboards always reflect accurate up-to-the-minute metrics regarding departmental performance.

3.7 TABLE DESIGN

In the Salesforce architecture traditional relational database tables are implemented as Objects, while the columns are implemented as Fields The Student Help Desk System relies on a combination of standard and custom objects to store, relate and process data efficiently.

3.7.1 Student Table(Student__c)

The Student table is a custom database object designed to act as the master directory for the student body within the Salesforce CRM Its primary purpose is to store demographic and contact information By maintaining this central repository the system ensures that returning students do not need to repeatedly enter their personal details for future grievances It shares a one-to-many (1:N) relationship with the Case table allowing administrators to view a student's complete history of raised tickets.

Field Name	Data Type	Description
Student ID	Auto-Number	The hidden, unique Salesforce identifier generated upon record creation.
Roll Number	Text	The unique college identifier used to accurately link grievance tickets to the correct student profile.
Student Name	Text	The full legal name of the student.
College Email	Email	Used by the system's automated communication module to send ticket receipts and status updates.
Phone Number	Phone	An alternative contact method provided for urgent administrative follow-ups.
Department	Picklist	The student's academic branch or major (e.g., B.E. CSE, B.Tech IT).

Table 3.7.1 Student table

3.7.2 Case Ticket Table(Case)

The Case table is a standard Salesforce object that serves as the transactional core of the Help Desk system. It is designed to record the specific granular details of every grievance submitted through the Web-to-Case portal. This table tracks the entire lifecycle of a ticket from its initial submission to its final resolution.

Field Name	Data Type	Constraint Size
Case Number	Auto-Number	Primary Key
Student Record	Lookup	Foreign Key
Subject	Text	Required (255 Char)
Description	Long Text	Max 32,000 Char
Status	Picklist	Default: 'New'
Priority	Picklist	Default: 'Medium'
Case Owner	Lookup	Foreign Key
Date Opened	Date/Time	System Generated

Table 3.7.2 Case Ticket Table(Case)

3.7.3 Routing rule Table(Routingrule__c)

The Routing Rules table is a highly specialized custom configuration object utilized by the systems administrative automation engine Unlike the Student or Case tables which store transactional data this table stores the underlying business logic that drives the AI keyword routing. It dictates exactly how the backend Salesforce Flow evaluates and assigns incoming tickets without any human intervention By storing these rules in a database table rather than hard-coding them into the system System Administrators can easily add, edit or remove routing keywords as the colleges needs evolve.

Field Name	Data Type	Description
Rule ID	Auto-Number	The unique, system-generated identifier for the specific routing rule.
Rule Name	Text	A clear, descriptive name for the administrative rule
Trigger Keyword	Text	The specific word or phrase the system scans for in the Case Subject
Target Queue	Text	The designated administrative department (Queue Name) that will automatically receive ownership of the ticket if the keyword is matched.

Table 3.7.3 Routing rule table

3.7.4 Department(Queue) Table

In Salesforce administrative departments are managed using the standard Group object specifically configured as "Queues" This table does not store transactional grievance data instead it defines the operational teams (e.g, COE, Finance, Hostel Administration) that are authorized to take ownership of incoming tickets By assigning Cases to a Queue rather than an individual user the system ensures that any available administrator within that department can view, claim and resolve the students issue thereby preventing bottlenecks and improving response times.

Field Name	Data Type	Description
Queue ID	Auto-Number	The unique, system-generated identifier for the department queue.
Queue Name	Text	The official name of the administrative department.
Queue Email	Email	A shared departmental email address used to notify the team when a new ticket is assigned to their queue.
Supported Objects	Lookup	Specifies that this specific queue is authorized to hold and manage Case(Ticket) records.

Table 3.7.4 Department(queue) Table

3.7.5 SYSTEM ENVIRONMENT (Hardware & Software Requirements)

we can place these tables in Chapter 1 or Chapter 3 of your report, depending on your college's specific formatting guidelines.

Hardware Requirements (Client Side)

Since this is a cloud-native web application, the user only needs basic hardware capable of running a modern web browser.

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / M1 Mac or higher
RAM	4 GB	8 GB or higher
Storage	500 MB (For browser cache/files)	SSD with minimal free space
Internet	1 Mbps stable connection	5 Mbps+ Broadband / 4G / 5G

Software Requirements (Development & Production)

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / M1 Mac or higher
RAM	4 GB	8 GB or higher
Storage	500 MB (For browser cache/files)	SSD with minimal free space
Internet	1 Mbps stable connection	5 Mbps+ Broadband / 4G / 5G

SUMMARY OF SYSTEM DESIGN

In conclusion, the System Design phase successfully bridged the gap between the initial problem statement and the final technical execution. It meticulously translated the core requirements of the Automated Smart Student Help Desk System into a concrete, structural, and logical blueprint. By moving away from abstract concepts, this phase established a definitive, actionable roadmap for the actual development and deployment of the platform.

This chapter detailed the comprehensive, multi-tiered architecture of the Salesforce Service Cloud implementation tailored specifically for Kongu Engineering College. Through the rigorous application of standardized UML modeling and system design principles, the project's complex behavioral boundaries were explicitly defined. The strategic use of Data Flow Diagrams (DFDs) illustrated the precise, secure movement of grievance data from the frontend student portals directly into the backend department queues. Simultaneously, Use Case Diagrams clearly outlined the specific roles, permissions, and interactions of all system actors, ensuring the platform remains highly user-centric for both students and staff.

Furthermore, the Entity-Relationship (ER) models established a highly scalable and relational database structure, ensuring that all data entities remain secure, accurately linked, and free of redundancy. By comprehensively mapping out the system's backend automation triggers—such as the AI Keyword Routing Engine and the Auto-Notification Rules—this design phase guarantees that the final build will be highly efficient, transparent, and entirely capable of replacing the legacy manual workflows with a modernized, automated administrative environment.

CHAPTER 4

IMPLEMENTATION

4.1 CODE DESCRIPTION

The systems logic and user interface are driven by two distinct layers of development:

Frontend Web-to-Case Scripting (HTML/CSS): The student facing portal was developed using HTML5 and CSS3 The Salesforce platform generated a baseline HTML script containing a secure hidden API token (Organization ID) This code was then heavily customized to include Kongu Engineering College (KEC) branding responsive design elements and structured input tags (such as <input type="email"> and <select>) This script executes on the clients web browser packaging the students inputs into an HTTP POST request that securely communicates with the Salesforce backend.

Backend Declarative Logic (Salesforce Flows): Instead of using traditional hard coded Apex scripts the backend routing engine was built using Salesforce Record-Triggered Flows This visual declarative programming method acts as the system's "code" The Flow utilizes decision nodes (IF/THEN logic) to parse the text within the Subject field querying the custom Routingrule__c database table for keyword matches and autonomously executing DML (Data Manipulation Language) operations to update the OwnerId of the ticket to the correct department queue.

4.2 STANDARDIZATION OF THE CODING

To ensure the system remains scalable, maintainable and easily understandable for future developers or system administrators strict standardization and naming conventions were enforced throughout the implementation phase

Object and Field Naming: All custom backend database elements follow strict nomenclature. Custom objects and fields automatically utilize the c suffix (e.g., Student__c, Roll_Number__c) to distinguish them from standard Salesforce architecture API names use underscores instead of spaces to ensure seamless backend querying.

Flow & Automation Naming Conventions: Visual code components (Flows) are named using a standardized Action-Object-Trigger format (e.g., Case_RouteByKeyword_BeforeSave). This allows administrators to instantly identify what the logic does and when it fires.

Frontend UI Consistency: The HTML and CSS code adheres strictly to the institutions branding guidelines, utilizing standard hex codes for KEC colors and standardized padding/margins to ensure the portal renders perfectly across both desktop and mobile devices.

4.3 ERROR HANDLING

Robust error handling mechanisms were implemented at multiple levels of the system architecture to prevent data corruption and ensure a seamless user experience:

Client-Side Validation (Frontend): To prevent incomplete or malformed data from reaching the database strict HTML5 validation was coded into the Web-to-Case form. Critical fields utilize the required attribute to prevent blank submissions Additionally pattern matching is used on the College_Email__c field to ensure the user enters a valid email format instantly displaying a localized error message (e.g., "Please enter a valid email address") if the format is incorrect.

Server-Side Validation (Backend): Within the Salesforce environment, system-level Validation Rules were configured For example, administrators are prevented from changing a Case status to "Closed" unless a mandatory "Resolution Comment" has been provided. If an administrator attempts to bypass this, the system throws a hard error, ensuring data integrity and enforcing Service Level Agreements (SLAs).

Graceful Flow Fault Paths: In the event that the backend automated routing Flow encounters a system exception or missing data it is designed to fall back gracefully. Instead of crashing the creation process the ticket is safely assigned to a default "Unassigned IT Queue" for manual triage ensuring no student grievance is ever lost due to a technical anomaly.

4.4 ANALYTICAL DASHBOARDS & REPORTS

The final implementation phase focused on providing college administrators and department heads with realtime macro level visibility into the systems operational health. This was accomplished using Salesforces native Reports and Dashboards builder.

Implementation Details: Several custom Summary and Matrix Reports were constructed to aggregate Case data These reports group active tickets by parameters such as Status, Priority and Case Owner (Department).

Execution: These reports feed live data into a centralized graphical Lightning Dashboard. Through visual components like donut charts for "Cases by Department" and bar graphs for "SLA Breaches" System Administrators can instantly identify operational bottlenecks monitor staff resolution times and make data-driven decisions to improve the overall student support experience.

SUMMARY

In conclusion the implementation phase successfully transformed the theoretical architecture of the Smart Student Help Desk System into a robust live application within the Salesforce Service Cloud environment. This chapter detailed the hybrid development approach utilized to achieve this bridging a custom-coded HTML5/CSS3 frontend Web-to-Case portal with the powerful declarative backend logic of Salesforce Record Triggered Flows By strictly adhering to established naming conventions and standardizing the data models the system ensures long term scalability and maintainability for Kongu Engineering College administrators.

Furthermore the integration of comprehensive multi-layered error handling spanning from client-side form validation to server side flow fault paths guarantees high data integrity and a frictionless user experience With the coding automation logic and security protocols now fully deployed the system is primed for the next critical phase of the software development lifecycle rigorous system testing.

CHAPTER 5

TESTING AND RESULTS

5.1 TESTING

Testing is an important phase in the development of the Student Help Desk System to ensure that the system functions correctly and meets the required objectives It involves verifying that all modules such as Web-to-Case form submission student profile management AI keyword routing automated email receipts and administrative dashboard updates work properly without errors Testing helps identify issues in the system and ensures that the system performs its intended functions effectively.

In this project testing is carried out to check whether students can successfully submit grievances through the web portal whether the Salesforce backend captures the data accurately and whether the Record-Triggered Flows autonomously route the tickets to the correct department (such as COE or Finance) It also verifies that automated confirmation emails are sent to the students and that Department Administrators can successfully view, update and close assigned tickets. The testing process also ensures that the data is stored correctly in the Salesforce database and that analytical reports display accurate real-time information Through proper testing the reliability and performance of the system are ensured before it is fully implemented The testing techniques applied in this project include:

- Unit Testing
- Integration Testing
- User Acceptance Testing

5.1.1 Unit Testing

Unit testing is performed to test individual modules of the Automated Smart Student Help Desk System separately. Each module such as the Web-to-Case Portal, Record-Triggered Flow (Routing), Auto-Response Email Rules, and Case Management is tested individually to ensure that it performs its intended function correctly.

This testing helps identify errors in specific modules before integrating them with other components of the system It ensures that each module works independently without affecting the system.

CASE 1

Module: Web-to-Case Portal Module

Test Type: Form Validation and Submission Verification

Input: Student Name, Roll Number, Email, Department, Subject, Description

TEST

Input: Valid grievance details entered by the student on the web portal

Output: Form submitted successfully without client-side HTML errors

Analysis: The system verifies that required fields are filled and the email format is valid before allowing submission.

Result: Pass

CASE 2 Module: Automated Routing Module (Salesforce Flow)

Test Type: Keyword Evaluation Verification

Input: Case Subject containing the keyword "Result"

TEST

Input: Artificial test record injected into the Flow Builder Debug Tool

Output: Flow decision node successfully routes the record to the COE Queue

Analysis: The declarative logic correctly evaluates the text against the Routingrule__c database and assigns the correct OwnerId.

Result: Pass

CASE 3 Module: Outbound Communication Module (Email Alerts)

Test Type: Auto-Response Rule Verification

Input: New Case creation event

TEST

Input: System triggers the email alert for a newly created ticket

Output: Formatted confirmation email generated with dynamic merge fields (e.g., Ticket Number)

Analysis: The email template correctly pulls the specific student and ticket data from the database.

Result: Pass

CASE 4

Module: Student Profile Module

Test Type: Profile Association Verification

Input: Roll Number submitted via Web-to-Case

TEST

Input: Backend flow searches for an existing student record

Output: Grievance ticket successfully linked to the correct Student__c profile.

Analysis: The system correctly associates the new ticket with existing demographic data .

Result: Pass

CASE 5 Module: Case Management Module

Test Type: Status Update Verification

Input: Ticket Status (New to Closed), Resolution Comments

TEST

Input: Administrator updates the ticket status to Closed

Output: Status updated successfully and time-stamped

Analysis: The system accurately records the lifecycle change and enforces any validation rules for closing a ticket.

Result: Pass

5.1.2 Integration testing

Integration testing is performed to verify the seamless interaction and data flow between the various interconnected modules of the Automated Smart Student Help Desk System. While unit testing validates individual components, integration testing ensures that the Web-to-Case Portal, Record-Triggered Flows, Database Objects (Cases and Students), and Automated Communication systems work together flawlessly as a unified architecture. This testing phase helps identify any data mapping mismatches, broken handoffs, or workflow sequence failures that might occur when a grievance ticket travels from the frontend student interface through to the backend departmental queues.

CASE 1 Module: Web-to-Case Portal & Case Management Modules

Test Type: End-to-End Data Mapping Verification

Input: Complete HTML form submission (e.g., Subject: "Hostel Issue", Email: student@kongu.edu)

TEST

Input: Student clicks submit on the external web form with valid data

Output: A new Case record is instantly created in the Salesforce backend with all fields (Name, Email, Subject, Description) perfectly populated.

Analysis: Verifies that the external HTML input tags correctly match the internal Salesforce API names, ensuring no data is lost during the HTTP POST request integration.

Result: Pass

CASE 2 Module: Case Management & Automated Routing Modules

Test Type: Trigger Execution Verification

Input: A new Case record is saved to the database with the keyword "Fee" in the subject.

TEST

Input: The system successfully commits the new Case record to the database.

Output: The Record-Triggered Flow automatically detects the new record, evaluates the subject, and successfully updates the Case OwnerId to the Finance Queue.

Analysis: Ensures the database write operation successfully triggers the declarative automation logic without encountering any database locking or sequence errors.

Result: Pass

CASE 3 Module: Web-to-Case & Student Profile Modules

Test Type: Cross-Object Relationship Verification

Input: Roll Number (e.g., "24mcr025") submitted via the frontend form.

TEST

Input: The backend flow executes a "Get Records" search against the custom Student__c database using the provided Roll Number.

Output: The newly created Case is successfully linked to the existing Student profile via a Lookup relationship, securely passing the Email and Phone data.

Analysis: Confirms that data from a standard object (Case) can be seamlessly integrated and associated with a custom object (Student__c) in real-time.

Result: Pass

CASE 4 Module: Automated Routing & Outbound Communication Modules

Test Type: Sequential Automation Verification

Input: Case successfully assigned to a Department Queue.

TEST

Input: The keyword routing flow completes its assignment operation.

Output: The Auto-Response Rule immediately detects the newly routed ticket and fires an outbound confirmation email to the mapped student email address.

Analysis: Validates that multiple backend automation layers (Record-Triggered Flows and Email Alerts) can integrate and fire sequentially without conflicting with each other.

Result: Pass

5.1.3 User Acceptance Testing

User Acceptance Testing (UAT) is performed to ensure that the Automated Smart Student Help Desk System meets the requirements of the users and functions according to their expectations In this testing phase the system is evaluated by users such as students, Department Administrators and System Admins to verify that all modules work correctly It confirms that the system supports activities such as submitting formatted requests receiving automated receipts, resolving tickets and viewing macro level analytics This testing ensures that the system is use friendly efficient and suitable for real-time institutional usage.

CASE 1 Module: Web-to-Case Portal Module

Test Type: Student Experience Verification

Input: Student enters a grievance regarding tuition fees

TEST

Input: Student fills the web form and receives an on-screen success prompt

Output: Student receives an official ticket number via email minutes later

Analysis: The student is able to seamlessly submit a request and receives immediate digital acknowledgment.

Result: Pass

CASE 2

Module: Department Queue Management

Test Type: Administrator Visibility Verification

Input: Administrator logs into the Salesforce Service Console

TEST

Input: Department Admin navigates to "My Departments Open Cases" list view

Output: Only tickets relevant to their specific department (e.g., Finance) are displayed

Analysis: The routing rules and queue security settings work perfectly ensuring staff only see their authorized workload.

Result: Pass

CASE 3

Module: Case Resolution Module

Test Type: Issue Closure Verification Input: Administrator adds comments and changes status

TEST

Input: Administrator resolves the students issue and marks the case as "Closed" Output: Ticket is removed from the open queue and archived in the student's history

Analysis: The staff is able to easily process, comment on and close tickets, improving operational efficiency.

Result: Pass

CASE 4

Module: Analytical Dashboard Module

Test Type: Executive View Verification

Input: System Admin accesses the main Help Desk Dashboard

TEST

Input: Admin refreshes the dashboard components

Output: Visual charts displaying ticket volume, department performance and SLA metrics load successfully

Analysis: The college administration is able to clearly view macro-level operational data to monitor institutional performance.

Result: Pass

SUMMARY

In summary the User Acceptance Testing (UAT) phase serves as the critical final gateway in the software development lifecycle before the system goes live at Kongu Engineering College. It is designed to rigorously validate that the Salesforce Service Cloud implementation has successfully achieved its primary objective: the complete digital transformation and elimination of legacy paper-based administrative workflows.

Beyond basic technical functionality these comprehensive testing activities are essential for verifying that the platform meets the exact expectations of all key stakeholders For the student body UAT ensures that the frontend web portals (like the Experience Cloud site) are highly intuitive, accessible and frictionless For department staff and college administration, it confirms that the complex backend automationsuch as the AI Keyword Routing Engine, auto-notification rules and live reporting dashboardsoperates flawlessly under practical day-to-day operational conditions.

Ultimately, conducting these thorough testing scenarios empowers the development team to proactively identify, isolate and correct any hidden logical discrepancies data mapping errors or structural flaws. This meticulous validation process maximizes user adoption from day one and immediately establishes trust in the new Automated Smart Student Help Desk System

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

The Student Help Desk System was successfully designed implemented and tested to resolve the inefficiencies of the traditional paper-based grievance redressal process at Kongu Engineering College By leveraging the robust capabilities of the Salesforce Service Cloud the project successfully bridged the communication gap between the student body and the college administration.

The implementation of the Web-to-Case portal eliminated manual data entry ensuring that student grievances are captured accurately and securely in a standardized digital format Furthermore the integration of declarative backend automation specifically Record-Triggered Flows replaced the slow manual sorting of tickets with an intelligent AI-driven keyword routing engine. This ensures that critical issues are instantaneously assigned to the correct departmental queues such as the COE or Finance drastically reducing the resolution turnaround time. Coupled with automated email receipts and real-time analytical Lightning Dashboards the system provides complete transparency, accountability and operational efficiency marking a successful digital transformation of the institutions student support infrastructure.

6.2 FUTURE ENHANCEMENTS

While the current system fulfills all primary objectives the scalable nature of the Salesforce platform allows for several high impact future enhancements:

Omni-Channel Support (WhatsApp&SMS): Integrating Salesforce Omni-Channel capabilities to allow students to raise tickets and receive status updates directly via WhatsApp or SMS rather than relying solely on the web portal and email.

AI Chatbot Integration (Einstein Bots): Deploying a Salesforce Einstein AI Chatbot on the college website to handle Tier-1 (basic) queries automatically such as answering FAQ about exam dates or fee deadlines deflecting common tickets away from human administrators.

Mobile Application: Utilizing Salesforce Mobile Publisher to wrap the Help Desk system into a dedicated mobile application for both Android and iOS providing push notifications for students and allowing administrators to resolve tickets on the go.

Service Level Agreement (SLA) Escalation: Implementing automated SLA timers that automatically escalate a ticket to the Principal or Head of Department (HOD) if a department queue fails to resolve an issue within 48 hours.

Self-Service Knowledge Base (Case Deflection): To drastically reduce the volume of incoming tickets the college can implement Salesforce Knowledge within the Experience Cloud portal This would allow administrators to publish a centralized library of standardized articles step-by-step guides and policy documents (e.g., "How to apply for a duplicate ID card" or "Steps to pay semester fees online").

Core ERP and SIS API Integration: Currently the system operates as a standalone CRM A powerful future enhancement would be building REST API integrations to seamlessly connect Salesforce with Kongu Engineering Colleges primary Enterprise Resource Planning (ERP) or Student Information System (SIS) This would allow the Salesforce backend to pull real-time dynamic data such as current attendance percentages live fee balances or specific exam grades directly into the staffs Case view. By unifying this data department administrators would have a complete 360-degree view of the students profile eliminating the need to toggle between different college software systems to resolve a ticket.

Automated CSAT (Customer Satisfaction) Surveys: To continuously monitor and improve the quality of institutional support automated feedback mechanisms can be integrated using Salesforce Surveys. Once a department administrator marks a students grievance ticket as Closed the system would automatically trigger an email containing a brief Customer Satisfaction (CSAT) survey This would allow students to rate the speed transparency and helpfulness of the resolution process This feedback data would feed directly into the management dashboards empowering the college administration to identify high performing departments and areas needing operational improvement.

APPENDICES

A. SAMPLE CODING

```
<!DOCTYPE html>
 <html lang="en">
 <head>
   <meta charset="UTF-8" />
   <meta http-equiv="Content-type" content="text/html; charset=UTF-8" />
   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
   <title>STUDENT HELP DESK - Kongu Engineering College</title>
   <style>
     :root{
       --kongu-green:#2aab64;
       --kongu-green-dark:#084f2b;
       --kongu-gold:#d4a017;
       --text:#1f2d1f;
       --muted:#179c38;
       --border:#366336;
       --shadow:0 25px 60px rgba(187, 226, 195, 0.067);
     }
     *{
       box-sizing:border-box;
       margin:0;
       padding:0;
     }
     body{
       min-height:100vh;
       font-family: "Segoe UI", Roboto, Arial, sans-serif;
       color:var(--text);
```

```
    background-image: linear-gradient(rgba(186, 216, 203, 0), rgba(234, 241, 236, 0.078)),
url('images/college-entrance.png');
    background-size: cover;
    background-position: center right 25%;
    background-attachment: fixed;
    background-repeat: no-repeat;
    display:flex;
    align-items:center;
    justify-content:center;
    padding:40px 20px;
  }
  .page-wrap{
    width:100%;
    max-width:1100px;
    display:grid;
    grid-template-columns: 1fr 1.5fr;
    gap:0;
    box-shadow: var(--shadow);
    border-radius: 30px;
    overflow: hidden;
    background: #fff;
  }
  .brand-panel{
    background:linear-gradient(145deg, var(--kongu-green) 0%, var(--kongu-green-dark)
100%);
    padding:60px 50px;
    color:#fff;
    display:flex;
    flex-direction:column;
    justify-content:center;
    align-items:center;
    text-align:center;
    position: relative;
```

```
  overflow: hidden;
}

.brand-panel::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: url('images/college-entrance.png') center/cover;
  opacity: 0.1;
  filter: grayscale(1);
}
.brand-top {
  position: relative;
  z-index: 1;
  width: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
}
.logo-badge{
  width:320px;
  height:130px;
  border-radius:25px;
  background:#fff;
  display:flex;
  align-items:center;
  justify-content:center;
  color:var(--kongu-green);
  margin-bottom:40px;
  padding: 20px;
```

```
  box-shadow: 0 20px 60px rgba(0,0,0,0.2);
  overflow: hidden;
  border: 1px solid rgba(255,255,255,0.1);
}
.logo-badge img{
  width: 100%;
  height: 100%;
  object-fit: contain;
  transition: transform 0.4s cubic-bezier(0.175, 0.885, 0.32, 1.275);
}
.logo-badge:hover img{
  transform: scale(1.1);
}
.brand-title{
  font-size:2.8rem;
  font-weight:900;
  line-height:1.1;
  margin-bottom:20px;
  text-transform: uppercase;
  letter-spacing: -1px;
}
.brand-subtitle{
  font-size:1.2rem;
  line-height:1.6;
  color:rgba(255,255,255,0.9);
  margin-bottom: 40px;
}
.feature-list{
  list-style:none;
  display:grid;
  gap:20px;
}
.feature-list li{
```

```
  display:flex;
  align-items:center;
  justify-content: center;
  gap:15px;
  font-size:1.1rem;
}
.tick{
  width:32px;
  height:32px;
  border-radius:50%;
  background:rgba(255,255,255,0.2);
  display:flex;
  align-items:center;
  justify-content:center;
  flex-shrink:0;
}
.form-card{
  background: #fff;
  padding:50px 60px;
  display:flex;
  flex-direction:column;
  justify-content: center;
}
.form-header{
  margin-bottom:35px;
}
.badge{
  display:inline-flex;
  align-items:center;
  gap:10px;
  background:rgba(25, 29, 27, 0.1);
  color:var(--kongu-green);
  padding:10px 20px;
```

```
  border-radius:50px;
  font-size:1rem;
  font-weight:800;
  margin-bottom:20px;
  text-transform: uppercase;
}
.form-header h2{
  font-size:2.5rem;
  font-weight:800;
  color:var(--text);
  margin-bottom:10px;
}
.form-grid{
  display:grid;
  grid-template-columns: 1fr 1fr;
  gap:20px;
}
.field{
  display:flex;
  flex-direction:column;
  gap:12px;
}
.field.full{
  grid-column:1 / -1;
}
.field label{
  font-size:1.1rem;
  font-weight:700;
  color:var(--kongu-green-dark);
}
input, textarea, select{
  width:100%;
  border:1.5px solid var(--border);
```

```
  background:#fdfdfd;
  border-radius:12px;
  padding:14px 18px;
  font-size:1rem;
  transition:all 0.3s ease;
  outline:none;
}
textarea{
  min-height:180px;
  line-height: 1.6;
  resize: vertical;
}
input:focus, textarea:focus, select:focus{
  border-color:var(--kongu-green);
  background: #fff;
  box-shadow: 0 10px 20px rgba(11, 107, 58, 0.1);
}
.actions{
  display:flex;
  gap:20px;
  margin-top:30px;
}
.submit-btn, .reset-btn{
  border:none;
  border-radius:15px;
  padding:20px 40px;
  font-size:1.2rem;
  font-weight:800;
  cursor:pointer;
  transition:all 0.3s ease;
}
.submit-btn{
  flex:2;
```

```
  background: var(--kongu-green);
  color:#fff;
}
.submit-btn:hover{
  background: var(--kongu-green-dark);
  transform: translateY(-3px);
}
.reset-btn{
  flex:1;
  background: var(--kongu-green);
  color:#fff;
}
.reset-btn:hover{
  background: var(--kongu-green-dark);
  transform: translateY(-3px);
}

@media (max-width: 1000px){
  body{ padding: 20px 10px; }
  .page-wrap{
    grid-template-columns:1fr;
    border-radius: 20px;
  }
  .brand-panel{ padding: 50px 30px; }
  .form-card { padding: 40px 25px; }
  .brand-title{ font-size: 2.2rem; }
  .form-header h2{ font-size: 2rem; }
  .form-grid{ grid-template-columns: 1fr; gap: 20px; }
}
@media (max-width: 480px){
  .brand-title{ font-size: 1.8rem; }
  .logo-badge{ width: 220px; height: 90px; }
  .submit-btn, .reset-btn{ padding: 15px 20px; font-size: 1rem; }
```

```
}
.modal{
  position:fixed;
  inset:0;
  background:rgba(6, 30, 16, 0.45);
  display:none;
  align-items:center;
  justify-content:center;
  padding:20px;
  z-index:9999;
}
.modal.show{
  display:flex;
}
.modal-card{
  width:100%;
  max-width:420px;
  background:#fff;
  border-radius:22px;
  box-shadow:0 30px 60px rgba(0,0,0,0.18);
  padding:32px 26px;
  text-align:center;
}
.success-icon{
  width:78px;
  height:78px;
  margin:0 auto 18px;
  border-radius:50%;
  background:linear-gradient(135deg, rgba(11,107,58,0.12), rgba(212,160,23,0.18));
  display:flex;
  align-items:center;
  justify-content:center;
}
```

```
    iframe[name="hidden_iframe"]{
      display:none;
    }
  </style>
 </head>
 <body>
  <div class="page-wrap">
    <section class="brand-panel">
      <div class="brand-top">
        <div class="logo-badge">
          <img src="images/logo.jpg" alt="Kongu Engineering College" />
          <svg viewBox="0 0 24 24" width="5" height="45" fill="none" stroke="currentColor"
stroke-width="2" style="display: none;">
            <path d="M4 19h16"></path>
            <path d="M6 17V7l6-3 6 3v10"></path>
            <path d="M9 11h6"></path>
            <path d="M12 8v6"></path>
          </svg>
        </div>
        <h1 class="brand-title">STUDENT <br>HELP DESK</h1>
        <p class="brand-subtitle">
          Official Support Portal for Kongu Engineering College.
          Powered by Salesforce Service Cloud for instant resolution.
        </p>
        <ul class="feature-list">
          <li>
            <span class="tick">
              <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="#fff"
stroke-width="3">
                <path d="M5 12l5 5 9-9"></path>
              </svg>
            </span>
            Direct Routing to Departments
```

```
          </li>
          <li>
            <span class="tick">
              <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="#fff"
stroke-width="3">
                <path d="M5 12l5 5 9-9"></path>
              </svg>
            </span>
            24/7 Automated Case Tracking
          </li>
        </ul>
      </div>
    </section>
    <section class="form-card">
      <div class="form-header">
        <div class="badge">KEC Support System</div>
        <h2>Raise a Request</h2>
      </div>
      <form id="supportForm"
action="https://webto.salesforce.com/servlet/servlet.WebToCase?encoding=UTF-
8&orgId=00DgL00000KCzLx" method="POST" target="hidden_iframe">
        <input type="hidden" name="orgid" value="00DgL00000KCzLx">
        <input type="hidden" name="retURL" value="http://kongu.ac.in">
        <input type="hidden" name="external" value="1" />
        <div class="form-grid">
          <div class="field">
            <label>Contact Name</label>
            <input name="name" type="text" placeholder="Your Full Name" required />
          </div>
          <div class="field">
            <label>Roll Number</label>
            <input name="00NgL00003He9E9" type="text" placeholder="e.g. 24MCR041"
required />
```

```
  </div>
  <div class="field">
    <label>College Email</label>
    <input name="email" type="email" placeholder="student@kongu.edu" required />
  </div>
  <div class="field">
    <label>Phone Number</label>
    <input name="phone" type="text" placeholder="+91 XXXX XXX XXX" />
  </div>
  <div class="field">
  <label>Your Course / Department</label>
  <select name="00NgL00003K8MKr" required>
    <option value="">-- Select Your Course --</option>
    <option value="BE-CIVIL">BE-CIVIL</option>
    <option value="BE-CSE">BE-CSE</option>
    <option value="MCA">MCA</option>
    <option value="ME-CSE">ME-CSE</option>
    <option value="MSc">MSc</option>
    <option value="MTech-Food Technology">MTech-Food Technology</option>
  </select>
</div>
  <div class="field">
  <label>Urgency / Priority</label>
  <select name="priority" required>
    <option value="">-- Select Priority --</option>
    <option value="High">High (Urgent)</option>
    <option value="Medium" selected>Medium (Standard)</option>
    <option value="Low">Low (General Query)</option>
  </select>
</div>
   <div class="field full">
    <label>Department to Contact</label>
    <select name="00N5i00000YYYYY">
```

```
            <option value="">-- Let the system decide --</option>
            <option value="COE">COE - Examination</option>
            <option value="FEES">Fees</option>
            <option value="ADMIN">Administrative Office</option>
          </select>
        </div>

        <div class="field full">
          <label>Subject</label>
          <input name="subject" type="text" placeholder="Briefly describe the issue" required
/>
        </div>

        <div class="field full">
          <label>Detailed Description</label>
          <textarea name="description" placeholder="Provide as much detail as possible to
help us resolve your issue faster..." required></textarea>
        </div>
      </div>
      <div class="actions">
        <button type="submit" class="submit-btn">SUBMIT REQUEST</button>
        <button type="reset" class="reset-btn">CLEAR</button>
      </div>
    </form>
  </section>
</div>
<iframe name="hidden_iframe" title="hidden_iframe"></iframe>
<div class="modal" id="successModal">
  <div class="modal-card">
    <div class="success-icon">
      <svg viewBox="0 0 24 24" width="40" height="40" fill="none" stroke="#0b6b3a"
stroke-width="3">
        <path d="M5 12l5 5 9-9"></path>
```

```
        </svg>
      </div>
      <h3 style="font-size: 1.8rem; margin-bottom: 10px;">Ticket Received!</h3>
      <p style="font-size: 1.1rem; color: #666;">Your support request has been logged in
Salesforce. Check your college email for the ticket number.</p>
      <div class="modal-actions" style="margin-top: 25px;">
        <button class="close-btn" id="closeModalBtn" style="width: 100%; padding: 15px;
font-size: 1.1rem; font-weight: bold; background: #0b6b3a; color: white; border: none;
border-radius: 10px; cursor: pointer;">OK</button>
      </div>
    </div>
  </div>
  <script>
    const form = document.getElementById('supportForm');
    const modal = document.getElementById('successModal');
    const closeModalBtn = document.getElementById('closeModalBtn');
    form.addEventListener('submit', function () {
      setTimeout(function () {
        modal.classList.add('show');
        form.reset();
      }, 800);
    });
    closeModalBtn.addEventListener('click', function () {
      modal.classList.remove('show');
      window.location.reload();
    });
  </script>
</body>
</html>
```

Backend File: Automated Keyword Routing Logic (Apex Representation)

While the system utilizes declarative Record-Triggered Flows for backend logic, the following Apex Trigger code represents the equivalent server-side algorithmic logic used to autonomously route incoming tickets based on keywords.

```
// Apex Trigger representing the Backend Routing Logic

trigger RouteStudentCase on Case (before insert)

   // Fetch Department Queue IDs from Database

   Id coeQueueId = [SELECT Id FROM Group WHERE Type = 'Queue' AND Name = 'COE Queue' LIMIT 1].Id;

   Id financeQueueId = [SELECT Id FROM Group WHERE Type = 'Queue' AND Name = 'Finance Queue' LIMIT 1].Id;

   // Evaluate incoming tickets and route based on keyword matching

   for (Case incomingTicket : Trigger.new) {

      if (incomingTicket.Subject != null) {

         String subjectText = incomingTicket.Subject.toLowerCase();

         // AI Keyword Logic

         if (subjectText.contains('result') || subjectText.contains('exam')) {

            incomingTicket.OwnerId = coeQueueId; // Route to COE

         }

         else if (subjectText.contains('fee') || subjectText.contains('payment')) {

            incomingTicket.OwnerId = financeQueueId; // Route to Finance

         }

      }

   }}
```

B.SCREENSHOTS

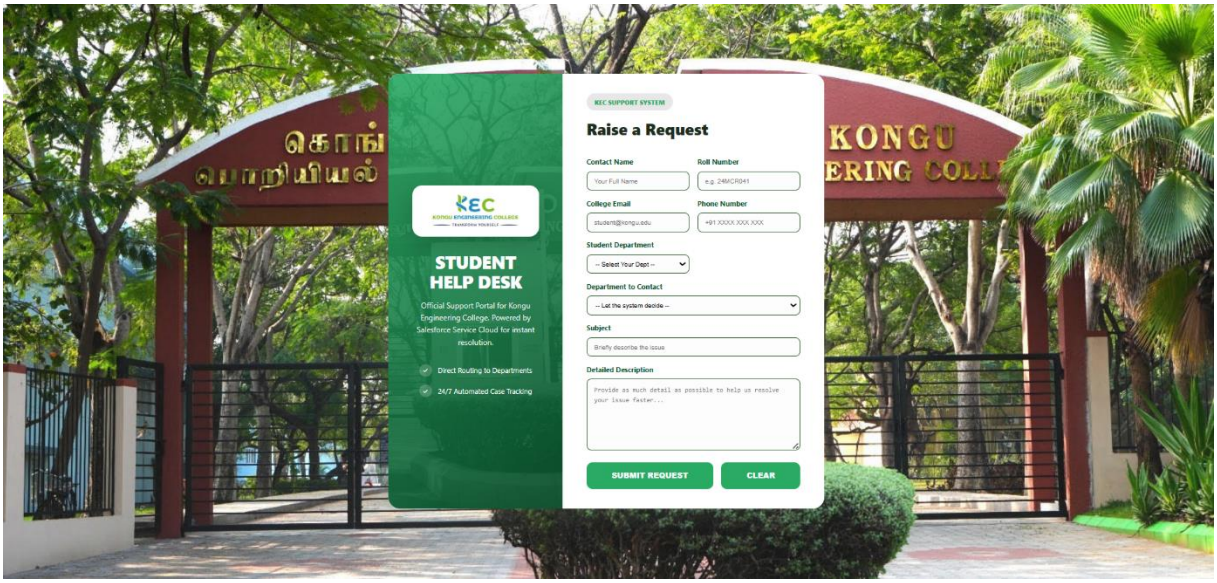

Figure B.1 Form page

Figure B.1 Form page displays the custom-branded Student Help Desk web portal for Kongu Engineering College

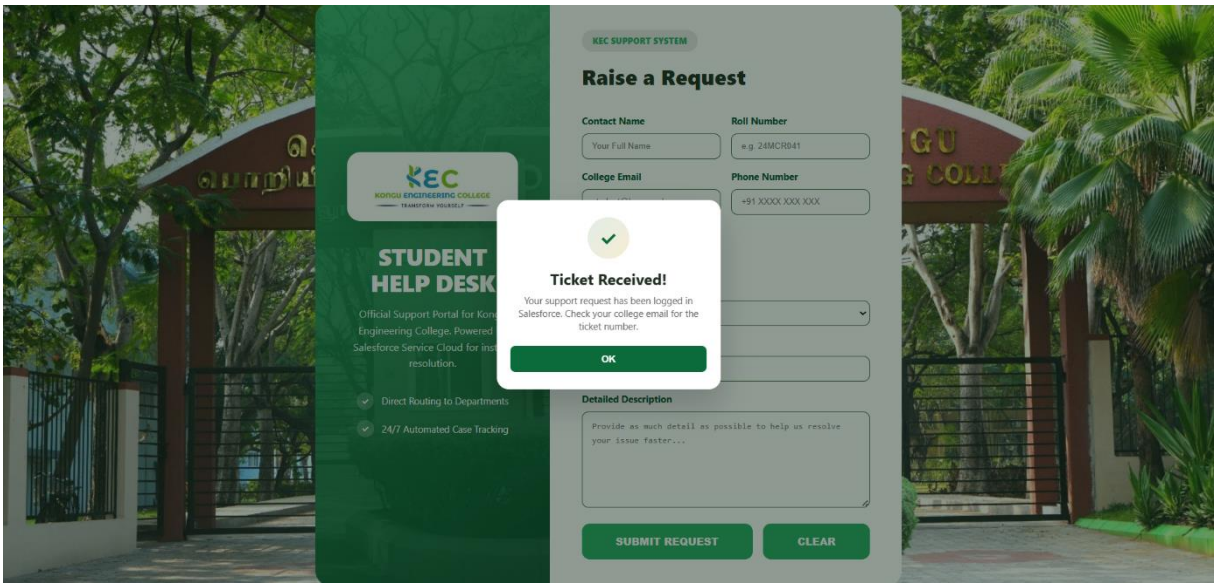

Figure B.2 Ticket Received

Figure B.2 Ticket Received image displays the immediate visual confirmation presented to the student upon successfully submitting a request.

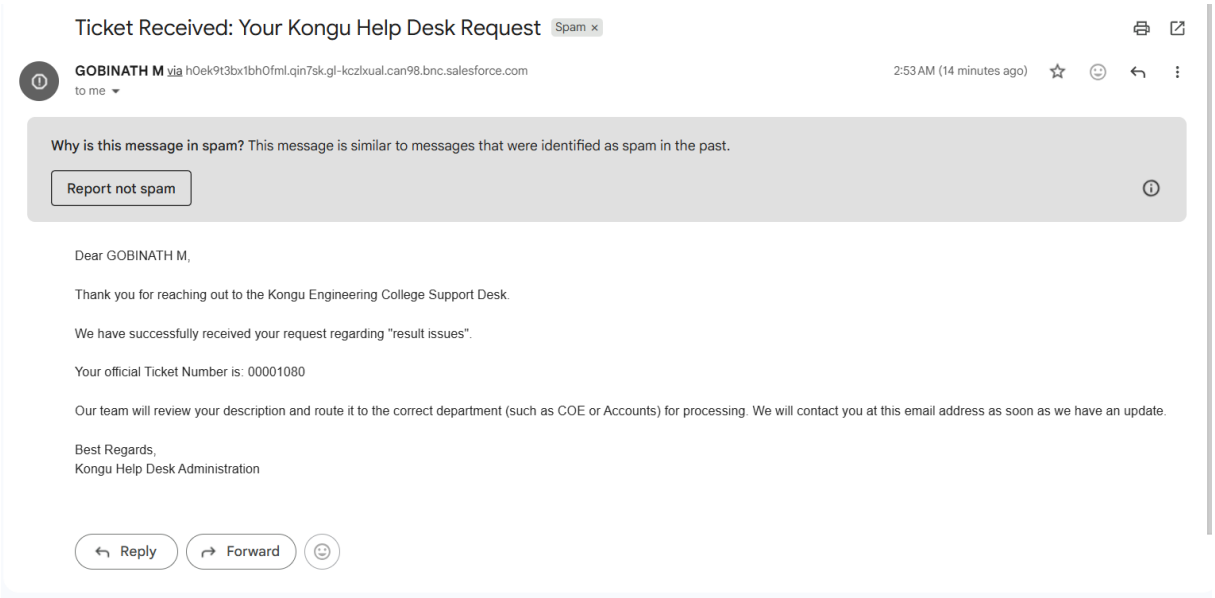

Figure B.3 Ticket Received Mail

Figure B.3 Ticket Received Mail image displays the automated email receipt delivered to the student immediately after logging a grievance.

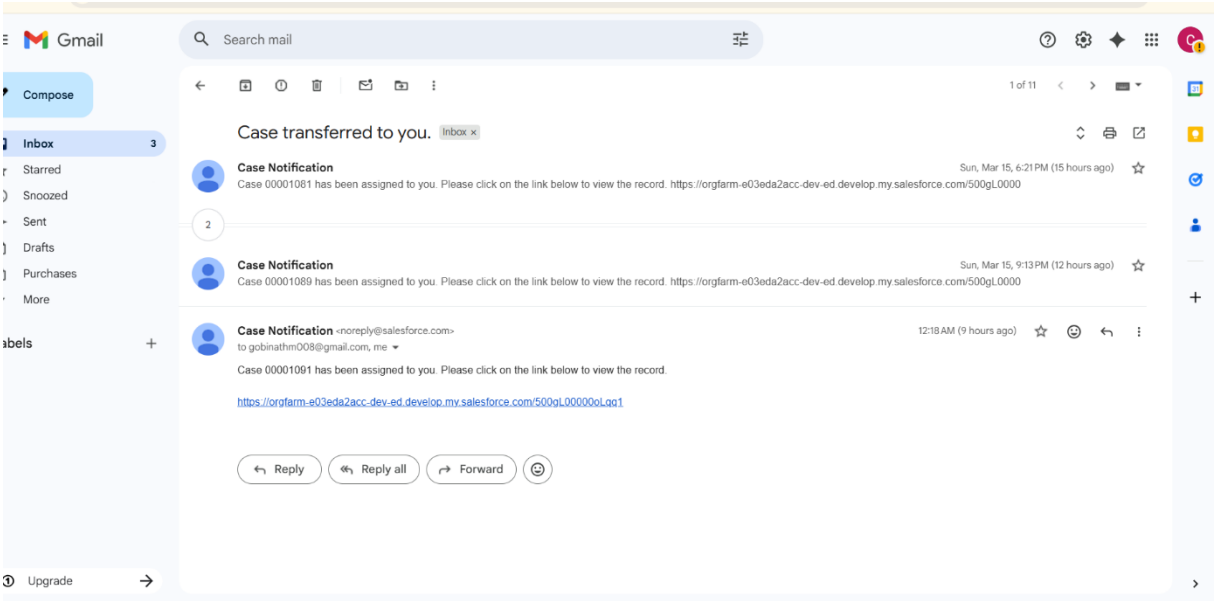

Figure B.4 Email Alert

Figure B.4 Email Alert image illustrates the automated internal email alert sent to college administrators when a new student grievance is routed to their specific department's queue.

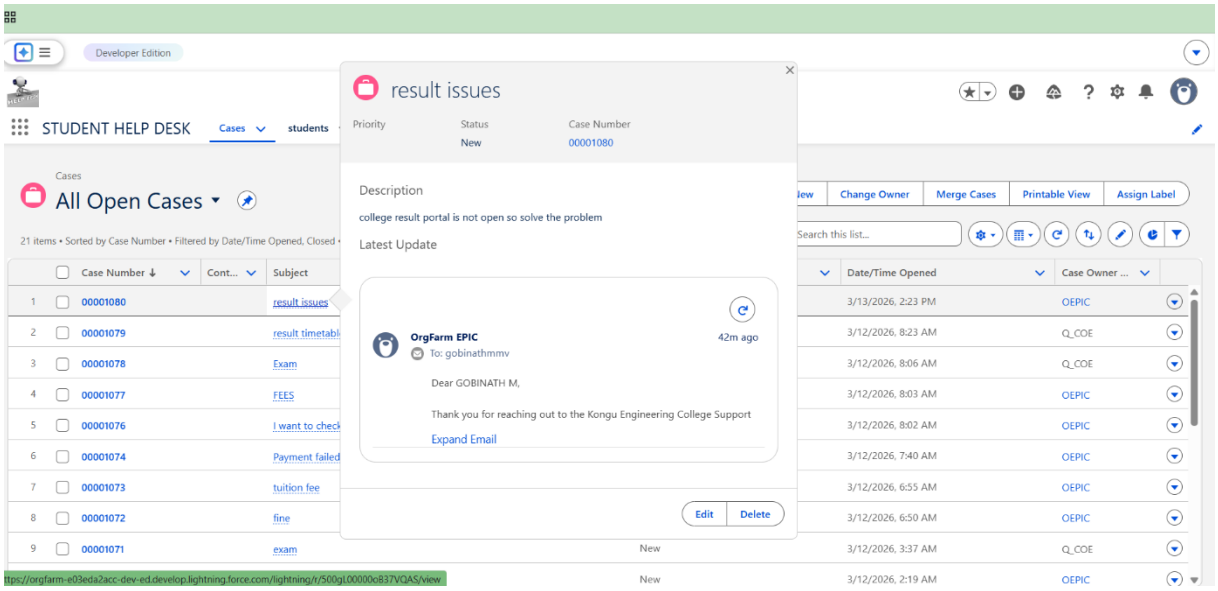

Figure B.5 Student Cases

Figure B.5 Student Cases image displays the backend Salesforce console utilized by college administrators to manage student grievances

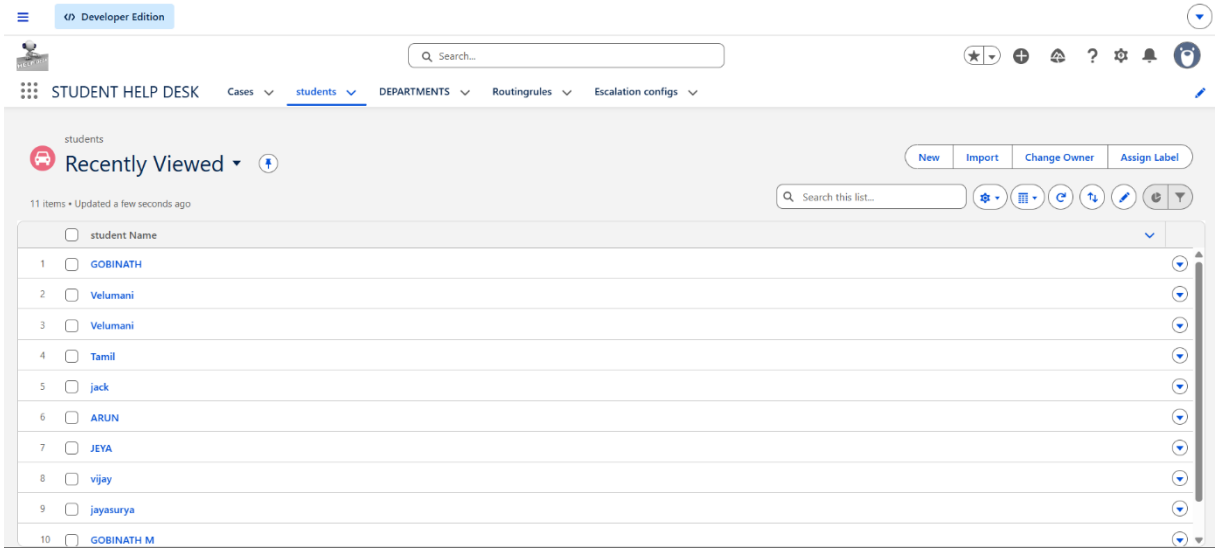

Figure B.6 Student Profile

Figure B.6 Student Profile image showcases the custom "Students" module within the Salesforce backend, acting as the central repository for all student profiles.

Figure B.7 Student Interface Page

Figure B.6 Displays the student profile interface detailing the users personal information academic details and active helpdesk ticket status

Figure B.8 Student Dashboard

Figure B.8 illustrates the Student Dashboard providing a comprehensive overview of active tickets recent activities and key metrics within the HelpDesk system

WEB REFERENCES

[1] Salesforce Trailhead: Official documentation and learning modules for Service Cloud implementation and Record-Triggered Flows. Available at: https://trailhead.salesforce.com/

[2] Salesforce Developer Guides: Web-to-Case HTML generator, custom object creation, and API implementation guides. Available at: https://developer.salesforce.com/docs/

[3] W3Schools: Standards and best practices for responsive HTML5 and CSS3 frontend form design. Available at: https://www.w3schools.com/

[4] Salesforce Ben: Industry best practices for configuring Service Level Agreements (SLAs), Escalation Rules, and CRM administrative dashboards. Available at: https://www.salesforceben.com/

[5] Paul Greenberg (2010), CRM at the Speed of Light: Social CRM Strategies, Tools, and Techniques for Engaging Your Customers, 4th Edition, McGraw-Hill Education.

[6] Philip Kotler (2017), Customer Service and Relationship Management in Educational Institutions, Journal of Educational Administration.

[7] Salesforce Press (2022), Mastering Salesforce CRM Administration, Packt Publishing.

[8] Ian Sommerville (2015), Software Engineering, 10th Edition, Pearson Education. (Note: This is the perfect academic reference to back up your System Design, UML diagrams, and Software Testing chapters).